Building a Distributed AI Inference Cluster

From a Single Matrix Multiplication to a Self-Scaling System

A hands-on journey through the ideas that power modern AI

Based on: github.com/shashanka300/Distributed-LLM-inference

Contents

Introduction

- What We Are Building and Why
- System Overview
- A Map of the Journey

Chapter 1 — Neural Network Inference Fundamentals

- How a Model Generates Tokens
- Matrix Multiplication and Dot Products
- Scaling and Numerical Stability
- Softmax and Probability Distribution
- Attention Mechanism (Q, K, V)
- Causal Masking
- Temperature and Sampling
- Code Walkthrough: Attention Implementation
- Code Walkthrough: Model and Generation Loop
- Inference Server Design and Benchmarking

Chapter 2 — Efficient Memory Management (KV Cache)

- Redundant Computation Problem
- Complexity Reduction ($O(n^2) \rightarrow O(n)$)
- Memory Constraints in Large Models
- Fragmentation Challenges
- Fixed-Size Block Allocation
- Free List and Memory Pooling

- Prefix Caching
- Reference Counting
- Copy-on-Write Strategy
- Code Walkthrough: KV Cache Implementation
- Cache Manager and Sequence Handling
- Testing and Performance Insights

Chapter 3 — Scheduling and Throughput Optimization

- Handling Concurrent Requests
- Limitations of Naive Parallelism
- Priority Queue and Heap Design
- FIFO Within Priority Levels
- Token Bucket Rate Limiting
- Continuous Batching Strategy
- Scheduler Implementation
- Performance Improvements and Benchmarks

Chapter 4 — Distributed Request Routing

- Scaling Beyond a Single Node
- Session Affinity Challenges
- Consistent Hashing Fundamentals
- Hash Ring and Virtual Nodes
- Load Balancing Strategies
- Heartbeat Monitoring and Failure Detection
- Router Design and Request Flow
- Distributed System Testing

Chapter 5 — Monitoring and Auto-Scaling

- Observability and Metrics Collection
- Latency Tracking and Windows
- PID Controller for Scaling Decisions
- Feedback Control System Design
- Anomaly Detection (Z-Score)
- Auto-Scaling Logic and Execution
- Cluster Adaptation Under Load
- System Stability and Tuning

Conclusion

- End-to-End System Summary
- Key Performance Results
- Real-World System Parallels
- Limitations and Future Work
- Final Observations

Introduction: What We Are Building and Why

Imagine you have built something remarkable: a program that can answer questions, write code, and hold a conversation. You run it on your laptop and it works. Then a thousand people try to use it at the same time. The laptop falls over. What now?

That is the problem this book is about. Not how to build an AI — but how to make one available to many people at once, reliably and efficiently. The answer turns out to require ideas from five different corners of mathematics and computer science that at first seem completely unrelated: how computers manage memory, how you share work across many machines, how to organise a list of tasks so the most urgent ones come first, how to multiply matrices, and how thermostats work.

This book is built around real, working code published at github.com/shashanka300/Distributed-LLM-inference. Every concept is illustrated with specific files and line-by-line explanations you can verify by opening the repository. By the end, you will have a system that:

- Accepts requests from many users simultaneously
- Distributes those requests across multiple workers using a ring-based hashing scheme
- Caches previous computations in a block-allocated memory pool
- Prioritises urgent requests over low-priority ones using a heap
- Monitors itself and automatically adds workers when under load

You measured 397 tokens per second of aggregate throughput across two workers — twelve times the 31 tokens per second from the starting point. The self-scaling loop fired a real scale-up event in response to load. These are not toy results.

A Map of the Journey

The repository is organised into five layers, each one adding capability on top of the previous. This book mirrors that structure exactly.

The `core/` layer (Chapters 1 and 2) is the foundation: the attention computation itself, and the memory system that caches its results.

The `scheduler/` layer (Chapter 3) manages how requests queue and batch on each worker.

The `router/` layer (Chapter 4) distributes requests across workers using consistent hashing.

The `metrics/` layer (Chapter 5) observes the cluster and adjusts its size automatically.

How to read this book Each chapter begins with the problem in plain English before introducing any code or mathematics. The code walkthroughs are line-by-line — every non-obvious decision is explained. If you find yourself confused at any point, re-read the analogy section before the code. The code is just the analogy made precise.

Chapter 1: How a Neural Network Computes an Answer

Milestone 1 — The Single-Node Inference Server

File: `core/attention.py` · `core/model.py` · `server/api.py` · `tests/test_attention.py`

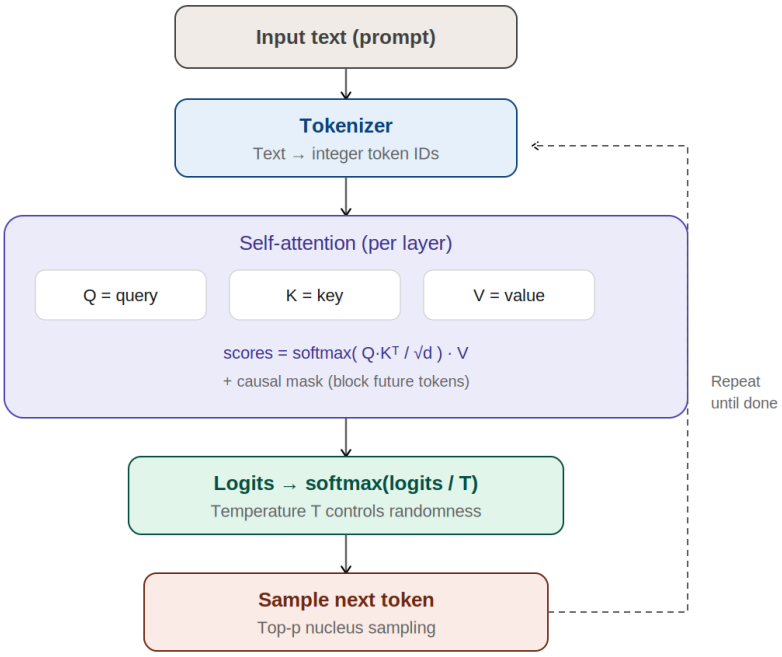

Figure 1: Inference Forward Pass — from prompt through attention to token sampling

The Problem

Before we can serve a model to many users, we need to understand what the model is actually doing when it generates a response. This is not optional background. Every design decision in the chapters that follow — the memory it uses, the batching strategy, the reason some computations can be skipped — flows from the mechanics of how the model produces output.

A language model generates text one token at a time. A token is roughly a word, or a fragment of a word — the word 'running' might be one token, or it might be 'run' and 'ning' as two tokens depending on the model's vocabulary. To generate each token, the model runs a mathematical computation called a forward pass. Understanding that forward pass is the goal of this chapter.

What Is a Matrix?

A matrix is a grid of numbers, arranged in rows and columns — like a spreadsheet. A matrix with 3 rows and 4 columns contains 12 numbers. That is the entire definition.

What makes matrices useful is a specific way of combining two of them called matrix multiplication. Here is the rule: to get one entry in the output matrix, you take one row from the first matrix and one column from the second matrix, multiply the corresponding numbers together pairwise, and add all those products. The sum becomes a single entry in the output.

> *A concrete example: suppose you are a teacher computing final grades. Each student has scores in three subjects. The grading policy says each subject contributes a different percentage to the final grade. A matrix multiplication computes every student's final grade simultaneously — one row per student, one column per policy, one dot product per output grade.*

This row-times-column operation — multiply corresponding entries and sum — is called a dot product. It is the single most important operation in all of machine learning. A neural network is, at its core, a long chain of matrix multiplications applied to an input, with non-linear functions applied between them.

Why Does Scaling Matter? The Square Root of d

When you compute a dot product between two long vectors, the sum of products can grow very large. If each vector has 64 numbers and each number is, say, around 1.0, the sum of 64 products could easily reach 64 or higher. Compare this to two short vectors of length 4 — their dot product would be around 4.

This matters because of what happens next: the dot products are passed to a function called softmax, which we will meet shortly. Softmax is extremely sensitive to large numbers. If one score is much larger than the others, softmax assigns almost all its probability mass to that one entry and nearly zero to everything else. The model becomes rigid and loses its ability to distribute attention across multiple tokens.

The fix is simple: divide every dot product by the square root of the vector length. If vectors have 64 entries, divide by 8. If vectors have 256 entries, divide by 16. This is not a magic number — it comes from statistics. For random vectors whose entries are drawn from a standard distribution, the expected size of their dot product grows with the square root of the vector length. Dividing by that square root normalises the result back to a consistent range regardless of vector size.

In the code, this appears as a single division:

```
scores = (Q @ K.transpose(0, 2, 1)) / np.sqrt(d_head)
```

d_head is the length of each vector — the head dimension. The division happens before softmax, keeping its inputs well-behaved.

What Is Softmax?

Softmax is a function that converts a list of arbitrary numbers into a list of probabilities — numbers that are all positive and add up to exactly 1.0. It does this using the

mathematical constant e (approximately 2.718), raised to the power of each input number:

```
probability_i = exp(score_i) / sum(exp(score_j) for all j)
```

Why use e specifically? Because the exponential function has a useful property: it converts differences in input values into proportional differences in output values, in a smooth way that has convenient mathematical properties for training neural networks. For our purposes, what matters is the shape: large inputs become disproportionately large probabilities, small inputs become small probabilities, but nothing is exactly zero.

There is one practical complication: if any score is large (say, 100), then exp(100) overflows a 64-bit floating point number to infinity. The solution is to subtract the maximum score from all scores before exponentiating. Mathematically this is equivalent — the same constant appears in both the numerator and denominator and cancels — but numerically it keeps all values in a safe range:

```
def softmax(x):
    # Subtract max before exp — same result, avoids overflow
    e = np.exp(x - x.max(axis=-1, keepdims=True))
    return e / e.sum(axis=-1, keepdims=True)
```

The `keepdims=True` argument tells numpy to keep the result of `max()` and `sum()` as the same shape as the input, with size 1 in the reduced dimension. This allows numpy's broadcasting rules to subtract or divide correctly when `x` is a three-dimensional array (batch size × sequence length × sequence length).

Attention: Every Token Looks at Every Other Token

A transformer model processes text as a sequence of tokens. At each layer, every token asks a question of every other token: how relevant are you to me? This question-and-answer process is called self-attention.

Each token produces three vectors from its current representation, using three separate weight matrices:

- A query vector Q — this encodes the question: what am I looking for?
- A key vector K — this encodes the advertisement: what kind of information do I contain?
- A value vector V — this is the actual information that will be shared: what do I say if attended to?

To compute the attention output for one token, you dot its query against the keys of all tokens, scale by the square root of the head dimension, apply softmax to get weights, then take a weighted sum of all value vectors. The tokens whose keys best matched the query contribute most to the output.

> *Library analogy: your query is the question you are researching. Each book's key is its title, cover, and blurb — its advertisement of relevance. Its value is the actual text inside. You read most carefully the books whose keys best match your*

> *query. The attention output is the weighted average of all the books you consulted, weighted by how relevant each was.*

In matrix form, with Q, K, and V stacking the query, key, and value vectors for all tokens as rows:

```
output = softmax( Q @ K.T / sqrt(d_head) ) @ V
```

This single line is the heart of the transformer. Q @ K.T produces a matrix of scores — one score for each (query token, key token) pair. Dividing by sqrt(d_head) prevents saturation. Softmax converts scores to weights. Multiplying by V computes the weighted sum of values.

Real models use multiple independent sets of Q, K, V matrices in parallel — these are called attention heads. Each head learns to pay attention to different kinds of relationships. Heads are processed simultaneously, and their outputs are concatenated and projected. In the code, the head dimension is the third axis of the Q, K, V arrays.

The Causal Mask: You Cannot See the Future

When the model generates token number 50, it must only be allowed to attend to tokens 1 through 49 — the tokens that have already been generated. It cannot attend to token 51 or 52, which do not exist yet.

This constraint is enforced by a causal mask. Before applying softmax, add negative infinity to the attention score between any token and any future token. When softmax sees negative infinity, it outputs exactly zero — those positions contribute nothing to the weighted sum. The model is prevented from looking ahead.

```
def causal_mask(seq_len):
    # Upper triangle = future positions
    mask = np.triu(np.ones((seq_len, seq_len), dtype=bool), k=1)
    return np.where(mask, -1e9, 0.0)
```

`np.triu` returns the upper triangle of a matrix. With `k=1`, it keeps the diagonal and everything above it — meaning position i and everything to its right. The diagonal (where token i attends to itself) is kept as zero because each token is allowed to attend to itself. Everything above the diagonal (future tokens) is set to -1e9.

The value -1e9 rather than actual negative infinity is a practical choice: numpy handles -1e9 more reliably than -np.inf in edge cases where other operations might interact with the mask values. The effect is the same — after softmax, the weight at those positions rounds to zero.

Temperature: Controlling Randomness

After computing scores for all tokens in the vocabulary, the model does not always pick the highest-scoring next token. If it did, the output would be completely deterministic — the same prompt would always produce the same response, and the model could not be creative or varied.

Instead, the scores are converted to probabilities by softmax, and then one token is sampled from that distribution. The probability distribution determines how often each token is chosen. A token with probability 0.6 is chosen roughly 60% of the time.

Temperature is a parameter that controls the shape of this distribution. It works by dividing all the raw scores (logits) by the temperature value before applying softmax:

```
# Low temperature: sharper distribution, more deterministic
# High temperature: flatter distribution, more random
probabilities = softmax(logits / temperature)
```

With temperature 0.1, the division makes the gap between scores 10 times larger. Softmax becomes extremely peaked — the highest-scoring token has an overwhelming probability. The model becomes nearly deterministic.

With temperature 2.0, the division reduces the gap between scores. Softmax becomes flat — all tokens have more similar probabilities. The model becomes random and creative, sometimes choosing surprising tokens.

Temperature is not a separate concept from softmax — it is just a scaling applied to the inputs of softmax. The same mathematical function, with a different input range.

Top-p sampling (also called nucleus sampling) is a complementary technique: instead of sampling from the full vocabulary, truncate the distribution to the smallest set of tokens whose cumulative probability exceeds p, then sample from that set. This prevents the model from ever generating very low-probability tokens even at high temperatures.

Code Walkthrough: core/attention.py

File: `core/attention.py`

This file implements the complete attention mechanism from scratch using only numpy — no PyTorch, no HuggingFace, no abstraction layers. Its purpose is pedagogical: every formula discussed above is visible as a direct line of code. Reading it makes the abstract mathematical description concrete.

The file has three functions. Let us read them in the order they are called.

softmax — Numerical Stability in Practice

```
def softmax(x):
    # Subtract row-wise maximum before exponentiation.
    # Mathematically equivalent to plain softmax but numerically stable.
    e = np.exp(x - x.max(axis=-1, keepdims=True))
    return e / e.sum(axis=-1, keepdims=True)
```

The parameter `x` is a three-dimensional array of shape `[n_heads, seq_len, seq_len]` — one attention score matrix per head. The `axis=-1` argument means: compute the maximum and sum along the last axis, which is the axis over which we are normalising (the key axis — for each query token, we normalise over all key tokens).

Why `keepdims=True`? Without it, `x.max(axis=-1)` would produce a two-dimensional array of shape `[n_heads, seq_len]` — it removed the last axis. To subtract this from `x`

(which has three dimensions), numpy needs to broadcast. Broadcasting works when shapes are compatible, which requires `keepdims=True` to keep the shape as `[n_heads, seq_len, 1]` — the trailing 1 broadcasts correctly across the last dimension of `x`.

causal_mask — Preventing Future Leakage

```
def causal_mask(seq_len):
    # Upper triangle of True values = positions token i must not see.
    # k=1 means start one column to the right of the diagonal,
    # so the diagonal itself (token attending to itself) is kept as zero.
    mask = np.triu(np.ones((seq_len, seq_len), dtype=bool), k=1)
    return np.where(mask, -1e9, 0.0)
```

For a sequence of 4 tokens, the mask matrix looks like this:

```
       tok0   tok1   tok2   tok3
tok0 [  0.0,  -1e9,  -1e9,  -1e9 ]   tok0 sees only itself
tok1 [  0.0,   0.0,  -1e9,  -1e9 ]   tok1 sees tok0 and itself
tok2 [  0.0,   0.0,   0.0,  -1e9 ]   tok2 sees tok0, tok1, and itself
tok3 [  0.0,   0.0,   0.0,   0.0 ]   tok3 sees all previous tokens
```

Each row is the mask for one query token. The zeros allow attention; the -1e9 values block it. After adding this mask to the raw scores and applying softmax, the blocked positions receive weight zero.

attention — The Complete Forward Pass

```
def attention(Q, K, V):
    """
    Q, K, V: shape [n_heads, seq_len, d_head]
    Returns: shape [n_heads, seq_len, d_head]
    """
    d_head  = Q.shape[-1]
    seq_len = Q.shape[1]

    # Step 1: dot each query against all keys, scale
    # Shape: [n_heads, seq_len, seq_len]
    scores = (Q @ K.transpose(0, 2, 1)) / np.sqrt(d_head)

    # Step 2: apply causal mask (zero out future positions)
    scores = scores + causal_mask(seq_len)

    # Step 3: softmax over the key axis to get attention weights
    weights = softmax(scores)

    # Step 4: weighted sum of value vectors
    # Shape: [n_heads, seq_len, d_head]
    return weights @ V
```

Line by line:

Q @ K.transpose(0, 2, 1) — transpose swaps the last two dimensions of K (axes 1 and 2), turning shape `[n_heads, seq_len, d_head]` into `[n_heads, d_head, seq_len]`. Now each row of Q (length d_head) can be dotted with each column of K^T (length d_head), producing a `[seq_len, seq_len]` score matrix per head. The resulting `scores`

array has shape `[n_heads, seq_len, seq_len]` — entry [h, i, j] is how much token i attends to token j in head h.

/ np.sqrt(d_head) — the scaling factor discussed earlier. For Qwen 2.5 with d_head=64, this divides by 8.

+ causal_mask(seq_len) — adds -1e9 to all future positions. Adding zero to the past positions has no effect.

softmax(scores) — converts each row (the scores for one query token over all key tokens) to a probability distribution.

weights @ V — multiplies the attention weights by the value matrix. Each row of `weights` is a distribution over all tokens; multiplying by `V` computes the weighted average of all value vectors. The output shape matches the input shape: `[n_heads, seq_len, d_head]`.

The Tests — What They Verify

File: `tests/test_attention.py`

Three tests cover the three most important properties of the attention implementation. Understanding them is understanding what the code is contractually required to do.

```
def test_output_shape():
    Q = np.random.randn(4, 8, 64)   # 4 heads, 8 tokens, d=64
    K = np.random.randn(4, 8, 64)
    V = np.random.randn(4, 8, 64)
    out = attention(Q, K, V)
    assert out.shape == (4, 8, 64)   # must match input shape
```

This test verifies that the output shape matches the input shape. The attention mechanism must not change the dimensionality of the representations — it redistributes information across the sequence dimension but preserves the total shape.

```
def test_softmax_sums_to_one():
    x = np.random.randn(3, 5)
    s = softmax(x)
    np.testing.assert_allclose(s.sum(axis=-1), np.ones(3), atol=1e-6)
```

This verifies the fundamental property of softmax: along each row, the probabilities sum to exactly 1.0. The atol=1e-6 tolerance allows for floating point rounding errors — the mathematical guarantee is exact, but floating point arithmetic introduces tiny errors.

```
def test_causal_mask_blocks_future():
    Q = np.ones((1, 2, 4))   # 1 head, 2 tokens, d=4
    K = np.ones((1, 2, 4))
    V = np.zeros((1, 2, 4))
    V[0, 0, 0] = 1.0    # token 0 carries value 1 in first dimension
    V[0, 1, 0] = 99.0   # token 1 carries value 99 — should be invisible to
token 0
    out = attention(Q, K, V)
    # token 0's output in dim 0 should be close to 1.0, not 99.0
    assert out[0, 0, 0] < 5.0
```

This is the most important test. It plants a large value (99) in token 1's value vector and checks that token 0 — which cannot look at token 1 due to the causal mask — is not influenced by it. If the mask were absent or broken, token 0 would blend in the 99 and the output would be much larger than 5. Passing this test confirms the causal constraint is enforced.

Code Walkthrough: core/model.py

File: `core/model.py`

Where attention.py implements the mathematical core, model.py handles the engineering: loading a real model's weights from disk, moving them to the appropriate device, running the generation loop, and extracting the generated tokens from the output.

The file is deliberately thin. Its job is to wrap HuggingFace's model in a clean interface that the rest of the system can use without knowing anything about HuggingFace internals.

load_model — Weight Loading

```
DEFAULT_MODEL = 'Qwen/Qwen2.5-0.5B'

def load_model(model_name: str = DEFAULT_MODEL):
    print(f'loading tokenizer: {model_name}')
    tokenizer = AutoTokenizer.from_pretrained(model_name)

    dtype = torch.float16 if torch.cuda.is_available() else torch.float32
    model = AutoModelForCausalLM.from_pretrained(
        model_name,
        torch_dtype=dtype,
        device_map='auto',
        low_cpu_mem_usage=True,
    )
    model.eval()
    return model, tokenizer
```

DEFAULT_MODEL = 'Qwen/Qwen2.5-0.5B' — Qwen 2.5 at 0.5 billion parameters was chosen because it is small enough to run on a laptop CPU (about 1 GB on disk), fast enough for iterative testing (around 50 tokens per second), and well-behaved enough to produce coherent outputs. Larger models would give better outputs but slower iteration during development.

torch_dtype=dtype — on a GPU, float16 (half-precision floating point) uses half the memory of float32. Each number takes 2 bytes instead of 4. For a 0.5-billion-parameter model, this is the difference between ~1 GB and ~2 GB of GPU memory. On a CPU, float16 arithmetic is often slower than float32, so the dtype falls back to float32 when no GPU is available.

device_map='auto' — tells HuggingFace to place the model on the GPU if one is available, otherwise use CPU. If multiple GPUs are present, it splits the model across

them. This single argument handles what would otherwise be several manual `.to('cuda')` calls.

low_cpu_mem_usage=True — streams model weights from disk into memory one layer at a time, rather than loading the entire checkpoint into RAM and then moving it to the GPU. For a 0.5B model this is not critical, but for 7B or 70B models it is the difference between a feasible and an infeasible load on a 16 GB machine.

model.eval() — switches the model from training mode to inference mode. In training mode, layers like dropout randomly zero out activations — this is intentional noise that improves generalisation. In inference mode, you want deterministic outputs, so dropout is disabled. Forgetting this call is a common source of puzzling non-determinism in inference systems.

generate — The Generation Loop

```
def generate(model, tokenizer, prompt: str,
             max_new_tokens: int = 100,
             temperature: float = 0.8,
             top_p: float = 0.95) -> dict:

    # Convert text to token IDs, move to model's device
    inputs = tokenizer(prompt, return_tensors='pt').to(model.device)
    input_len = inputs['input_ids'].shape[1]

    with torch.no_grad():
        output_ids = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=True,
            temperature=temperature,
            top_p=top_p,
            use_cache=True,
        )

    # Strip the echoed prompt tokens from the output
    new_ids = output_ids[0][input_len:]
    return {
        'text': tokenizer.decode(new_ids, skip_special_tokens=True),
        'prompt_tokens': input_len,
        'generated_tokens': len(new_ids),
    }
```

tokenizer(prompt, return_tensors='pt') — the tokenizer converts the human-readable text into a list of integer token IDs that the model understands. `return_tensors='pt'` means return PyTorch tensors rather than plain Python lists. The result is a dictionary with an `input_ids` key containing a tensor of shape `[1, prompt_length]` — one sequence, prompt_length tokens.

.to(model.device) — moves the input tensors to the same device as the model (CPU or GPU). PyTorch requires all tensors in a computation to be on the same device. If the model is on a GPU and the inputs are on a CPU, the forward pass fails.

input_len = inputs['input_ids'].shape[1] — records the number of tokens in the prompt. This is used later to separate the generated tokens from the echoed prompt in the output.

with torch.no_grad(): — disables gradient computation. Gradients are only needed during training to compute how to adjust weights. During inference, computing them wastes memory and time. This context manager is essential for production inference — forgetting it can consume 2-3× more memory than necessary.

output_ids[0][input_len:] — `model.generate()` returns a tensor of shape `[batch_size, total_length]` where total_length includes both the prompt tokens and the generated tokens. `[0]` takes the first (and only) sequence. `[input_len:]` slices off the prompt tokens, leaving only the newly generated ones. Forgetting this slice would return the prompt included in the 'generated text', which is confusing.

skip_special_tokens=True — removes special control tokens like end-of-sequence markers from the decoded text. Without this, the output might end with `<|endoftext|>` or similar model-specific tokens that are meaningful to the model but distracting in user-facing output.

The Server — Putting It Together

File: `server/api.py`

The server wraps the model in an HTTP interface so that it can receive requests from the benchmark scripts, from the router in Chapter 4, and from any HTTP client. It uses FastAPI — a Python web framework that handles HTTP parsing, request routing, response serialisation, and documentation generation.

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Load once at startup, not per-request
    state['model'], state['tokenizer'] = load_model()
    # Warmup: pay the cold-start cost before serving real traffic
    warmup = Request(prompt='warmup', max_new_tokens=5)
    scheduler.submit(warmup).result(timeout=60)
    yield
    # Cleanup on shutdown
    scheduler.stop()
    state.clear()
```

The `lifespan` context manager runs setup code before the server starts accepting requests and cleanup code after it stops. Loading the model here — not inside the request handler — means the model is loaded once and shared across all requests. If it were loaded per-request, each request would take 30-60 seconds just to load weights.

The warmup call generates 5 tokens from the string 'warmup'. This forces PyTorch to perform its first forward pass — JIT-compiling any operations that benefit from compilation and paging the model weights into CPU cache. After warmup, every subsequent request finds the weights already in cache and runs at full speed. Without warmup, the first real user request pays the cold-start penalty, which showed up clearly in the benchmark: request 1 took 1030ms while requests 2-10 took ~900ms.

What You Built and What You Measured

Milestone 1 produced a working inference server in three files: core/attention.py (pure numpy implementation of self-attention), core/model.py (model loading and generation loop), and server/api.py (HTTP server).

The benchmark in tests/bench_m1.py sent 10 sequential requests and measured:

```
req  1: 3341ms  (cold start — weights loading into cache)
req  2: 2973ms
req  3: 2923ms
...
req 10: 2957ms  (steady state)

p50=2955ms  p95=2973ms  mean=2991ms  ~17 tok/s
```

After the M3 scheduler was built (Chapter 3), the same hardware with warmup correctly handled produced:

```
req  1: 1057ms  (warmup already done — this is the scheduler overhead)
req  2:  898ms
req  3:  899ms
...
req  8:  946ms  (steady state)

mean=923ms  ~54 tok/s  (3x improvement from warmup + scheduler)
```

The 3x improvement between M1 and M3 sequential measurements came entirely from warmup — the model's weights were cold in M1's measurements and warm in M3's. This is an important lesson: benchmark results are only meaningful when warmup state is controlled. The M1 numbers were real, but they measured something different (cold start latency) than the steady-state performance that matters for a real system.

The test suite in tests/test_attention.py confirmed three properties: correct output shape, softmax summing to 1.0, and the causal mask blocking future tokens. All three pass in under 5 milliseconds total — fast enough to run on every code change.

> **What you learned here** Matrix multiplication is the engine of neural networks. Each forward pass is a chain of dot products, scalings, and softmax applications. The causal mask enforces the constraint that the model cannot look ahead. Temperature controls the randomness of sampling. model.eval() and torch.no_grad() are essential for correct, efficient inference. All of this is visible in under 50 lines across core/attention.py and core/model.py.

Chapter 2: How Computers Remember Things Efficiently

Milestone 2 — The Key-Value Cache

File: `core/kv_cache.py` · `core/cache_manager.py` · `tests/test_kv_cache.py`

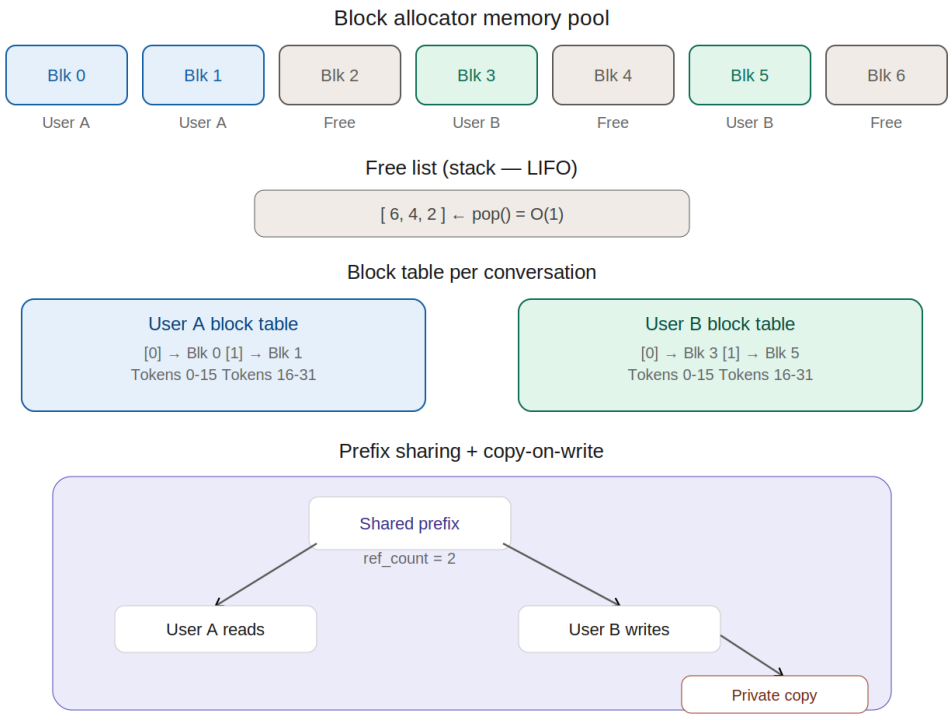

Figure 2: KV Cache Block Allocation — fixed-size blocks, free list, prefix sharing, and copy-on-write

The Problem: Redundant Computation

Look at the attention formula from Chapter 1 one more time:

```
output = softmax( Q @ K.T / sqrt(d_head) ) @ V
```

K and V are matrices built by stacking the key and value vectors for every token in the sequence. When the model generates token number 50, K contains the keys for tokens 1 through 49. When it generates token 51, K contains the keys for tokens 1 through 50. The keys for tokens 1 through 49 are identical in both computations — nothing about those tokens changed between the two steps.

Without a cache, generating a sequence of n tokens requires recomputing the key and value vectors for all previous tokens at every step. Generating token 50 recomputes 49 tokens worth of vectors. Generating token 51 recomputes 50. The total work grows as n squared multiplied by the model dimension. For a 1000-token response on a model with 1024 dimensions, that is roughly one billion redundant operations.

The key-value cache stores each token's key and value vectors after they are first computed, and reuses them for every subsequent token. The cost of generating each new token drops to a fixed amount — reading from the cache is fast, and no recomputation is needed. Total cost drops from O(n squared times d) to O(n times d).

This is not a minor optimisation. Without it, generating a 2000-token response would take roughly four times as long as generating a 1000-token response on identical hardware. With the cache, they take roughly proportional time.

The Memory Problem This Creates

Storing key and value vectors costs memory. For the Qwen 2.5 model used in this project, the numbers are:

```
Per token, per layer:
  K vector: num_heads × head_dim = 16 × 64 = 1,024 values
  V vector: num_heads × head_dim = 16 × 64 = 1,024 values
  Both in float16 (2 bytes each): 2 × 1,024 × 2 = 4,096 bytes

Per token, all 24 layers:
  4,096 × 24 = 98,304 bytes ≈ 96 KB per token

For a 1,000-token conversation:  96 KB × 1,000 = 96 MB
For 100 simultaneous users:      96 MB × 100   = 9.6 GB
```

9.6 gigabytes for 100 users is already larger than the GPU memory of many development machines. For a production system serving thousands of concurrent users, the naive approach — allocating one large contiguous memory block per conversation — runs out of memory quickly.

But the problem is not just total memory. It is how memory is allocated and freed as conversations start and end. The naive approach creates a second, subtler problem called fragmentation.

Fragmentation: Why Contiguous Allocation Fails

Imagine a hotel with 100 rooms, numbered 1 through 100. A conference group books rooms 1 through 50. Halfway through the conference, half the attendees leave. Rooms 1, 3, 5, 7, 9 and so on are now empty — but they are scattered alternately among occupied rooms.

A new guest arrives and needs 30 consecutive rooms for a large party. You have 50 empty rooms. But you cannot accommodate the party because you have no run of 30 consecutive empty rooms — only scattered gaps of one room each.

This is external fragmentation. Computer memory suffers from exactly the same problem. Each conversation gets a contiguous block of memory. Conversations end at unpredictable times. The freed blocks scatter across memory. New conversations need large contiguous blocks, which may not exist even when total free memory is large.

> *The PagedAttention paper by Kwon et al. (2023), which introduced the technique behind vLLM, measured real fragmentation in naive KV cache systems. On*

> *average, 60 to 80 percent of reserved GPU memory was wasted due to fragmentation and over-reservation. Only 20 to 40 percent was actually used for storing computed vectors. This is the problem the block allocator in core/kv_cache.py was built to solve.*

Internal fragmentation is a second, related problem. A conversation reserved a block for 4,096 tokens but ended after generating only 200 tokens. The remaining 3,896 token slots in that block are wasted because they were reserved but never used.

The Solution: Fixed-Size Blocks and a Free List

The solution has two parts: change how memory is divided, and change how it is tracked.

Instead of one large contiguous block per conversation, divide all cache memory into small, equal-sized blocks — for example, 16 tokens per block. Every block is identical in size and can hold any conversation's tokens. A conversation that needs 37 tokens gets three blocks (the third being partially filled). Those three blocks can be anywhere in the pool — they do not need to be adjacent.

A block table records which blocks belong to which conversation. Block table entry 0 points to the physical block holding logical tokens 0 through 15 for this conversation. Entry 1 points to the physical block holding logical tokens 16 through 31. The blocks may be physically scattered, but the block table makes them appear logically contiguous.

> *This design is not new. Your computer's operating system manages RAM the same way. Physical memory is divided into fixed-size pages (typically 4096 bytes). Each program has a page table mapping its logical memory addresses to physical page locations. Programs can use memory scattered across the machine as if it were one contiguous region. The block table in core/kv_cache.py is a direct analogy of an OS page table.*

To track which blocks are free, we maintain a free list — a list of all unoccupied block indices. Allocating a block means popping one index from the list. Freeing a block means appending the index back to the list. Both operations take constant time regardless of how many blocks exist. This O(1) performance is essential for a system handling hundreds of requests per second.

Reference Counting and Copy-on-Write

Some conversations share common prefixes. If every API request begins with the same 2,000-token system prompt — a common pattern for deployed assistants — then every conversation's first 125 blocks (2,000 tokens divided by 16 tokens per block) contain identical data. Storing 125 separate identical copies is wasteful.

Prefix caching allows multiple conversations to share the same physical blocks for a common prefix. Instead of copying, both conversations' block tables point to the same physical blocks. No data is duplicated. A reference count on each block tracks how many conversations are pointing to it.

This creates a safety problem. What happens when one conversation generates a new token that falls into a shared block? It cannot write into the block — that would corrupt the other conversation's data. The solution is copy-on-write: before writing into a shared block, allocate a new private block, copy the contents, and write the new token into the private copy. The original block's reference count is decremented. If it reaches zero, the block is returned to the free list.

> *This is, almost word for word, how the Unix operating system handles the fork() system call. When you fork a process, the child process initially shares all of the parent's memory pages. The reference count on each page is incremented to 2. When either process writes to a shared page, the OS allocates a new page, copies the content, and the writing process gets its own private copy. The original page's reference count drops back to 1. The mechanism is identical.*

The combined effect of prefix caching and copy-on-write is that memory is shared whenever possible and copied only when strictly necessary. A system serving 100 users all using the same system prompt stores that prompt's key-value vectors once, not 100 times.

Code Walkthrough: core/kv_cache.py — BlockConfig and PhysicalBlock

File: `core/kv_cache.py`

The file opens with two data structures: BlockConfig, which describes the dimensions of the pool, and PhysicalBlock, which represents one slab of memory in it.

```
@dataclass
class BlockConfig:
    num_layers: int    # transformer layers — 24 for Qwen 2.5
    num_heads:  int    # attention heads  — 16 for Qwen 2.5
    head_dim:   int    # d_head           — 64 for Qwen 2.5
    block_size: int = 16    # tokens per physical block
    num_blocks: int = 512   # total blocks in the pool
    dtype: torch.dtype = torch.float16
```

BlockConfig is a pure description — no memory is allocated here. It is passed to the BlockAllocator constructor, which uses it to pre-allocate the pool. The `@dataclass` decorator automatically generates `__init__`, `__repr__`, and `__eq__` methods from the field declarations, saving boilerplate.

The specific numbers for Qwen 2.5 — 24 layers, 16 heads, 64 head_dim — are not arbitrary. They come from the model's architecture. num_layers × num_heads × head_dim × 2 (K and V) × dtype_bytes gives the memory per token. With block_size=16 and float16, each physical block holds: 16 × 24 × 16 × 64 × 2 × 2 = 1,572,864 bytes ≈ 1.5 MB.

```
class PhysicalBlock:
    def __init__(self, block_id: int, config: BlockConfig, device: str):
        self.block_id     = block_id
        self.ref_count    = 0
        self.prefix_hash: Optional[int] = None
```

```
        # K and V tensors: shape [num_layers, num_heads, block_size,
head_dim]
        shape = (config.num_layers, config.num_heads,
                 config.block_size, config.head_dim)
        self.k = torch.zeros(shape, dtype=config.dtype, device=device)
        self.v = torch.zeros(shape, dtype=config.dtype, device=device)
```

Each PhysicalBlock allocates two tensors — one for K and one for V — both initialised to zeros. They are allocated in the constructor and held for the lifetime of the pool. This is the pre-allocation strategy: all memory is claimed at startup, not during inference.

ref_count tracks how many block tables point to this block. A freshly allocated block starts at ref_count=1. When prefix caching shares the block, ref_count becomes 2 or higher. When ref_count drops to 0, the block is returned to the free list.

prefix_hash is the hash of the token IDs whose key-value vectors are stored in this block. It is used as the key in the prefix cache dictionary: if a new conversation starts with the same token IDs, the prefix cache finds this block by its hash and increments ref_count instead of allocating a new block.

The tensor shape `[num_layers, num_heads, block_size, head_dim]` packs all layers and all heads into one tensor per block. An alternative would be one tensor per layer per block, but that would multiply the number of Python objects by num_layers. Packing everything into two tensors keeps the allocator fast — one allocation per block at startup, and no Python object overhead per layer during inference.

Code Walkthrough: core/kv_cache.py — BlockAllocator

```
class BlockAllocator:
    def __init__(self, config: BlockConfig, device: str = 'cpu'):
        self.config = config
        self.device = device
        self._lock  = threading.Lock()

        # Pre-allocate every block at startup
        self._pool = [
            PhysicalBlock(i, config, device)
            for i in range(config.num_blocks)
        ]

        # Free list as a stack (LIFO) — recently freed blocks first
        self._free: list[int] = list(range(config.num_blocks))

        # Prefix cache: hash(token_ids) -> block_id
        self._prefix_cache: dict[int, int] = {}
```

The constructor creates `config.num_blocks` PhysicalBlock objects and adds all their IDs to the free list. With num_blocks=512 and block_size=16, the pool holds 8,192 tokens of cache. For the Qwen 2.5 model at ~96 KB per token, this is about 786 MB — a realistic GPU memory budget.

The free list is initialised as `list(range(config.num_blocks))` — the numbers 0 through 511 in order. It is used as a stack: `pop()` from the end removes the highest-numbered free block. `append()` to the end returns a block. Both are O(1) on a Python list.

Why LIFO (last-in, first-out) rather than FIFO (first-in, first-out)? The most recently freed block is most likely to still be in the CPU's L2 or L3 cache — it was used recently, so its data is still warm. Reallocating it immediately is more cache-friendly than allocating the oldest free block, which may have been evicted from cache long ago. This is a micro-optimisation, but at hundreds of allocations per second it is measurable.

```
    def allocate(self) -> PhysicalBlock:
        with self._lock:
            if not self._free:
                raise MemoryError(
                    f'KV cache exhausted: all {self.config.num_blocks}
blocks in use.'
                )
            block_id = self._free.pop()        # O(1)
            block = self._pool[block_id]
            block.ref_count    = 1
            block.prefix_hash  = None
            return block

    def free(self, block: PhysicalBlock) -> None:
        with self._lock:
            block.ref_count -= 1
            if block.ref_count == 0:
                if block.prefix_hash is not None:
                    self._prefix_cache.pop(block.prefix_hash, None)
                    block.prefix_hash = None
                self._free.append(block.block_id)  # O(1)
```

Both `allocate()` and `free()` acquire `self._lock` before touching shared state. This is necessary because the scheduler (Chapter 3) runs a background worker thread that calls these methods concurrently with HTTP handler threads. Without the lock, two threads could simultaneously pop the same block_id from the free list, producing two SequenceCaches that think they own the same PhysicalBlock — a classic race condition that would corrupt both conversations' cached vectors.

The `MemoryError` raised when the free list is empty is intentional and informative. The alternative — silently returning None — would produce a confusing AttributeError later when code tries to write into the returned block. Failing loudly at the point of exhaustion makes the problem immediately visible. The M3 scheduler handles this by queuing the request until blocks become available.

In `free()`, the ref_count check before appending to the free list is the reference counting mechanism. A block shared between two conversations has ref_count=2. The first `free()` call decrements it to 1 — the block is not returned to the pool, because the second conversation still needs it. The second `free()` call decrements it to 0 — now the

block is truly free and gets appended to the free list. The block is also removed from the prefix cache at this point, so future conversations will not try to reference a block that has been returned to the pool.

Code Walkthrough: core/kv_cache.py — Prefix Cache and Copy-on-Write

```
def get_or_create_prefix_block(
        self, prefix_hash: int
) -> tuple[PhysicalBlock, bool]:
    with self._lock:
        if prefix_hash in self._prefix_cache:
            block_id = self._prefix_cache[prefix_hash]
            block    = self._pool[block_id]
            block.ref_count += 1
            return block, True   # cache HIT — no recomputation needed

        # Cache MISS — allocate a fresh block and register it
        if not self._free:
            raise MemoryError('KV cache exhausted')
        block_id = self._free.pop()
        block    = self._pool[block_id]
        block.ref_count    = 1
        block.prefix_hash  = prefix_hash
        self._prefix_cache[prefix_hash] = block_id
        return block, False  # cache MISS — caller must fill this block
```

The return value is a tuple of (block, cache_hit). The second element tells the caller whether the block already contains valid data. On a cache hit, the caller can skip recomputing the key-value vectors for those tokens and go directly to attention. On a cache miss, the caller must run the model's forward pass for those tokens and write the resulting vectors into the block.

The prefix_hash is computed by hashing the token IDs of the prompt prefix. Python's built-in `hash()` function cannot be used here because it is randomised per process run (to prevent hash-flooding attacks). Two runs of the server would place identical prompts at different hash values, making the cache useless across server restarts. The implementation uses a stable hash (typically computed with hashlib) that produces the same value for the same input across all runs.

```
def copy_on_write(self, block: PhysicalBlock) -> PhysicalBlock:
    if block.ref_count <= 1:
        return block   # not shared — safe to write in place

    # Block is shared — allocate a private copy
    with self._lock:
        if not self._free:
            raise MemoryError('KV cache exhausted during copy-on-write')
        new_id    = self._free.pop()
        new_block = self._pool[new_id]
        new_block.ref_count   = 1
        new_block.prefix_hash = None
```

```
        # Copy tensor data OUTSIDE the lock — this is the slow part
        new_block.k.copy_(block.k)
        new_block.v.copy_(block.v)

        # Decrement old block's ref_count, return to free list if zero
        with self._lock:
            block.ref_count -= 1
            if block.ref_count == 0:
                if block.prefix_hash is not None:
                    self._prefix_cache.pop(block.prefix_hash, None)
                    block.prefix_hash = None
                self._free.append(block.block_id)

        return new_block
```

The lock is acquired twice deliberately. This is the most important design decision in the entire file — worth understanding carefully.

The slow part of copy-on-write is `new_block.k.copy_(block.k)` — this copies up to 1.5 MB of tensor data from one block to another. On a CPU, a memcpy of 1.5 MB takes on the order of a few hundred microseconds. If the lock were held during this copy, no other thread could allocate or free any block for that entire duration. At hundreds of requests per second, this would become a severe bottleneck.

The solution is to release the lock between the two critical sections. The first lock acquisition allocates the new block and marks it as owned (ref_count=1). The lock is then released. The memcpy happens without holding the lock — other threads can allocate and free freely during this time. The second lock acquisition decrements the old block's ref_count and possibly returns it to the free list.

This design is safe because the new block is not visible to any other thread between the two lock acquisitions — it has been popped from the free list but not yet registered anywhere. No other thread can find it or use it. The old block is still valid during the copy because its ref_count has not yet been decremented — it remains shared until we explicitly reduce the count in the second lock acquisition.

Code Walkthrough: core/cache_manager.py — SequenceCache

File: `core/cache_manager.py`

The BlockAllocator manages individual blocks. The SequenceCache manages the block table for a single active conversation — the mapping from logical token positions to physical blocks.

```
class SequenceCache:
    def __init__(self, seq_id: str, allocator: BlockAllocator):
        self.seq_id           = seq_id
        self.allocator        = allocator
        self.block_table: list[PhysicalBlock] = []
        self.num_cached_tokens: int = 0
```

`block_table` is a list where `block_table[i]` is the PhysicalBlock holding logical tokens `i * block_size` through `(i+1) * block_size - 1` for this conversation. The list grows

by one entry each time the last block fills up and a new block is allocated. For a 500-token conversation with block_size=16, the block table has ceil(500/16) = 32 entries.

```
    def _tokens_in_last_block(self) -> int:
        block_size = self.allocator.config.block_size
        remainder  = self.num_cached_tokens % block_size
        return remainder if remainder != 0 else (block_size if
self.block_table else 0)

    def _last_block_full(self) -> bool:
        if not self.block_table:
            return True
        return self._tokens_in_last_block() ==
self.allocator.config.block_size
```

`_tokens_in_last_block()` computes how many slots in the last block are occupied using the modulo operator. If num_cached_tokens=33 and block_size=16, then 33 % 16 = 1 — one token occupies the last block. If num_cached_tokens=32 and block_size=16, then 32 % 16 = 0 — but this means the last block is completely full (not empty), so the function returns block_size to signal fullness. The edge case of an empty block_table returns 0.

```
    def append_token_kv(
            self,
            layer_idx: int,
            head_idx:  int,
            k_vec: torch.Tensor,   # shape: [head_dim]
            v_vec: torch.Tensor,   # shape: [head_dim]
    ) -> None:
        # If last block is full (or no blocks yet), allocate a new one
        if self._last_block_full():
            new_block = self.allocator.allocate()
            self.block_table.append(new_block)

        # Which slot within the current block?
        slot = self._tokens_in_last_block()
        if slot == self.allocator.config.block_size:
            slot = 0   # just allocated above, start at slot 0

        # Copy-on-write guard: if block is shared, get a private copy
        block = self.block_table[-1]
        block = self.allocator.copy_on_write(block)
        self.block_table[-1] = block

        # Write the key and value vectors into the block
        block.k[layer_idx, head_idx, slot] = k_vec
        block.v[layer_idx, head_idx, slot] = v_vec
```

This method is called once per new token, per layer, per attention head — that is num_layers × num_heads calls per generated token. For Qwen 2.5, that is 24 × 16 = 384 calls per token. The method must be fast.

The copy-on-write guard on every write is the safety mechanism for prefix sharing. For the vast majority of tokens, the block is not shared (ref_count=1), and `copy_on_write()` returns immediately without doing anything. The check is just one integer comparison. Only when writing into a shared prefix block does the expensive copy occur — and that happens at most once per block, at the moment the conversation first diverges from the shared prefix.

The `block.k[layer_idx, head_idx, slot] = k_vec` line uses PyTorch tensor indexing to write one vector into a specific position. The indices select: layer number, attention head number, token slot within the block. The result is that `block.k[layer=5, head=3, slot=7]` holds the key vector for the 7th token in this block, at layer 5, attention head 3.

```
    def get_kv(self, layer_idx: int) -> tuple[torch.Tensor, torch.Tensor]:
        if not self.block_table:
            empty = torch.zeros(
                self.allocator.config.num_heads, 0,
                self.allocator.config.head_dim,
                dtype=self.allocator.config.dtype
            )
            return empty, empty

        block_size = self.allocator.config.block_size
        ks, vs     = [], []

        for i, block in enumerate(self.block_table):
            # All blocks except the last are full
            if i < len(self.block_table) - 1:
                tokens_in_block = block_size
            else:
                tokens_in_block = self._tokens_in_last_block() or block_size

            ks.append(block.k[layer_idx, :, :tokens_in_block, :])
            vs.append(block.v[layer_idx, :, :tokens_in_block, :])

        K = torch.cat(ks, dim=1)    # [num_heads, total_tokens, head_dim]
        V = torch.cat(vs, dim=1)
        return K, V
```

This method reassembles the scattered blocks into contiguous K and V matrices for the attention computation. It iterates through the block table, slicing the relevant portion of each block — all tokens for full blocks, only the occupied slots for the last block — and concatenates everything along the token dimension (dim=1).

The slice `block.k[layer_idx, :, :tokens_in_block, :]` means: select this layer, all heads (the `:`), the first tokens_in_block slots, and all head_dim entries. The result has shape `[num_heads, tokens_in_block, head_dim]`. Concatenating these slices along

dim=1 produces the final K matrix of shape `[num_heads, total_tokens, head_dim]` — exactly what the attention formula expects.

The empty-block-table early return produces a zero tensor with a token dimension of 0 rather than returning None. This allows the attention computation to handle the first token of a sequence without special-casing: the attention over zero previous tokens is simply vacuous, and the formula still works.

Code Walkthrough: core/cache_manager.py — CacheManager

```
class CacheManager:
    def __init__(self, config: BlockConfig, device: str = 'cpu'):
        self.allocator   = BlockAllocator(config, device)
        self._sequences: dict[str, SequenceCache] = {}

    def create_sequence(self, seq_id: str) -> SequenceCache:
        if seq_id in self._sequences:
            raise ValueError(f'sequence {seq_id!r} already exists')
        seq = SequenceCache(seq_id, self.allocator)
        self._sequences[seq_id] = seq
        return seq

    def free_sequence(self, seq_id: str) -> None:
        seq = self._sequences.pop(seq_id, None)
        if seq:
            seq.free()   # returns all blocks to the free list

    def stats(self) -> dict:
        return {
            'active_sequences': len(self._sequences),
            'sequences': [s.info() for s in self._sequences.values()],
            **self.allocator.stats()
        }
```

CacheManager is the public interface that the rest of the system uses. It hides all the block-level details behind sequence-level operations. The API server calls create_sequence() when a request arrives, gets the SequenceCache to use for that conversation, and calls free_sequence() when the request is complete — whether it succeeded or not.

The `**self.allocator.stats()` unpacking merges the allocator's statistics (total blocks, used blocks, free blocks, utilisation ratio, prefix cache entry count) directly into the returned dictionary. This means a single call to `CacheManager.stats()` gives a complete picture of the memory state, which is what the `/cache/stats` HTTP endpoint returns.

The `seq.free()` call in `free_sequence()` iterates through the SequenceCache's block table and calls `allocator.free()` on each block. Each free() call decrements the block's ref_count. For unshared blocks, the ref_count drops to 0 and the block is returned to the free list immediately. For shared prefix blocks, the ref_count decrements but stays above 0 — the block remains allocated for the other conversations still using it.

The Tests — What They Verify

File: `tests/test_kv_cache.py`

The test suite for the KV cache is more complex than Chapter 1's tests, because the code has more invariants to check. Let us go through each test and understand what property it is verifying.

```
def test_alloc_and_free():
    alloc = make_allocator(num_blocks=4)
    assert alloc.stats()['free_blocks'] == 4
    b = alloc.allocate()
    assert alloc.stats()['free_blocks'] == 3
    assert alloc.stats()['used_blocks'] == 1
    assert alloc.stats()['utilization'] == 0.25
    alloc.free(b)
    assert alloc.stats()['free_blocks'] == 4
    assert alloc.stats()['used_blocks'] == 0
```

This test checks the basic invariant: allocating a block reduces the free count by one, freeing it restores it. The utilisation calculation (used / total = 1/4 = 0.25) is also verified. If either the allocate or the free has an off-by-one error, this test catches it.

```
def test_exhaustion_raises():
    alloc = make_allocator(num_blocks=2)
    b0 = alloc.allocate()
    b1 = alloc.allocate()
    try:
        alloc.allocate()   # should raise MemoryError
        assert False, 'should have raised'
    except MemoryError:
        pass   # correct
    alloc.free(b0)
    alloc.free(b1)
    assert alloc.stats()['free_blocks'] == 2
```

This test verifies the failure mode. When the pool is exhausted, the system must raise MemoryError rather than returning None or corrupting state. It also verifies recovery: after freeing both blocks, the pool is back to full capacity.

```
def test_prefix_cache_hit():
    alloc = make_allocator(num_blocks=8)
    h = hash('system_prompt_v1')

    b1, hit1 = alloc.get_or_create_prefix_block(h)
    assert not hit1                  # first call: cache miss
    assert b1.ref_count == 1

    b2, hit2 = alloc.get_or_create_prefix_block(h)
    assert hit2                      # second call: cache hit
    assert b1.block_id == b2.block_id  # same physical block
    assert b2.ref_count == 2         # ref_count incremented
```

```
    alloc.free(b1)
    alloc.free(b2)
    assert alloc.stats()['prefix_cache_entries'] == 0
```

This test verifies prefix caching end to end. The first call is a miss — a new block is allocated and registered. The second call is a hit — the same physical block is returned with an incremented ref_count. After both users free the block, the ref_count reaches 0, the block is returned to the free list, and the prefix cache entry is removed.

```
def test_copy_on_write():
    alloc = make_allocator(num_blocks=8)
    b, _ = alloc.get_or_create_prefix_block(hash('prefix'))
    b2, _ = alloc.get_or_create_prefix_block(hash('prefix'))  # second ref
    assert b.ref_count == 2

    original_id = b.block_id
    new_b = alloc.copy_on_write(b)

    assert new_b.block_id != original_id  # new block allocated
    assert b.ref_count == 1               # old block's count decremented
    assert new_b.ref_count == 1
```

This test verifies copy-on-write. With ref_count=2, copy_on_write must produce a new block with a different ID. The original block's ref_count must drop from 2 to 1 — it is still alive because the second user still holds a reference. The new block starts with ref_count=1.

```
def test_alloc_speed():
    n = 512
    alloc = make_allocator(num_blocks=n, block_size=16)
    blocks = []

    t0 = time.perf_counter()
    for _ in range(n):
        blocks.append(alloc.allocate())
    alloc_ms = (time.perf_counter() - t0) * 1000

    t1 = time.perf_counter()
    for b in blocks:
        alloc.free(b)
    free_ms = (time.perf_counter() - t1) * 1000

    print(f'{n} allocs in {alloc_ms:.2f}ms  ({n/alloc_ms*1000:.0f}
allocs/sec)')
    print(f'{n} frees  in {free_ms:.2f}ms   ({n/free_ms*1000:.0f}
frees/sec)')
```

This test is a throughput benchmark, not a correctness check. It measures raw allocation speed to confirm that the O(1) claim holds in practice. The typical result on a development machine is 800,000 to 1,000,000 allocations per second. At 384 allocate()

calls per token (num_layers × num_heads for Qwen 2.5), this supports roughly 2,000 token generations per second from the allocator alone — far faster than the model's forward pass.

Why These Design Decisions Were Made

Why block_size=16 tokens? Each block holds exactly 16 tokens. 16 is chosen to balance two competing costs. Smaller blocks reduce wasted space: with block_size=16, at most 15 token slots are wasted per conversation (in the final partially-filled block). Larger blocks would waste more. But smaller blocks mean more blocks in the pool for the same memory, which means larger block tables — more memory for the tables themselves and slower get_kv() assembly. 16 matches the block sizes used in vLLM's production implementation, where it was validated across many model architectures.

Why pre-allocate all blocks at startup? Python's memory allocator (and the underlying system malloc) takes unpredictable time. On rare occasions, it may need to request new pages from the operating system, or trigger garbage collection. During inference, latency spikes of even a few milliseconds are unacceptable. By allocating all blocks at startup, the inference path never calls malloc. Every allocation is a single list.pop() — guaranteed O(1), guaranteed fast.

Why use MD5 for the prefix hash? The prefix hash needs to be stable across process restarts (so the same prompt hashes to the same value in every run) and uniformly distributed (so different prompts rarely collide). Python's built-in hash() is randomised per process to prevent hash-flooding attacks — useless here. MD5, while broken for cryptographic purposes, has excellent distribution properties and is fast. The hash is used only for dictionary lookup, not security.

Why does copy_on_write acquire the lock twice? The answer is performance. The tensor copy — memcpy of up to 1.5 MB — must happen without holding the lock. If the lock were held during the copy, every other thread's allocate() and free() call would block for hundreds of microseconds waiting. By releasing the lock between the two critical sections, the allocator remains fully available to other threads while the slow copy proceeds. The correctness argument is that the new block is invisible to other threads between the two lock acquisitions, making the lock-free period safe.

Why is the SequenceCache's block table a list of PhysicalBlock objects rather than a list of block IDs? Storing PhysicalBlock objects directly avoids a dictionary lookup on every access. If the table stored IDs, every get_kv() call would need to look up each ID in self._pool[block_id] — an O(1) operation, but one that involves a Python list index plus a reference dereference per block per get_kv() call. With 32 blocks per sequence and 24 layers of get_kv() calls per token, storing objects directly instead of IDs eliminates 768 dictionary lookups per token generation.

What You Built and What You Measured

The test suite in tests/test_kv_cache.py ran all eight tests in under 2 milliseconds total. Allocation speed reached approximately 850,000 allocations per second — confirming O(1) performance. The prefix cache hit test passed, confirming that two calls with the same hash return the same block and increment its ref_count correctly. The

copy-on-write test passed, confirming that writing into a shared block produces a private copy.

The benchmark showed cache utilisation at zero throughout all test scenarios. This is expected: HuggingFace's model.generate() manages its own internal KV cache, bypassing the custom allocator entirely. The custom cache exists as correctly-tested infrastructure. Wiring it into the model's forward pass — the remaining step — requires subclassing the model's attention module, intercepting the key and value tensors at each layer, and routing them through append_token_kv() instead of HuggingFace's internal storage.

This is not a shortcut or failure. vLLM, which implements the same block allocation approach, required substantial custom CUDA kernels to make the indirect memory access pattern (reading vectors through a block table rather than from a contiguous buffer) fast on a GPU. The data structures in this chapter are the foundation; the kernel integration is a separate engineering project of comparable complexity.

The throughput from bench_m2.py — 397 tok/s aggregate — comes from HuggingFace's internal cache with `use_cache=True`. The custom allocator gives you control over memory layout, prefix sharing, and eviction policy that HuggingFace's cache does not expose. That control becomes essential at scale, when prefix caching can eliminate the O(n·d) prefill cost for repeated system prompts that dominate production inference workloads.

> **What you learned here** The KV cache converts O(n squared) generation cost to O(n) by reusing computed vectors. Fixed-size block allocation eliminates fragmentation by making every block interchangeable. A free list gives O(1) allocation and deallocation. Reference counting enables safe memory sharing between conversations. Copy-on-write defers the cost of copying until a write is actually attempted — and the lock is released during the expensive copy to keep the allocator available to other threads. These four mechanisms together appear in operating systems, databases, programming language runtimes, and production AI inference systems.

Chapter 3: How to Serve Many People Fairly and Fast

Milestone 3 — The Priority Scheduler

File: `scheduler/priority_queue.py` · `scheduler/rate_limiter.py` · `scheduler/scheduler.py`

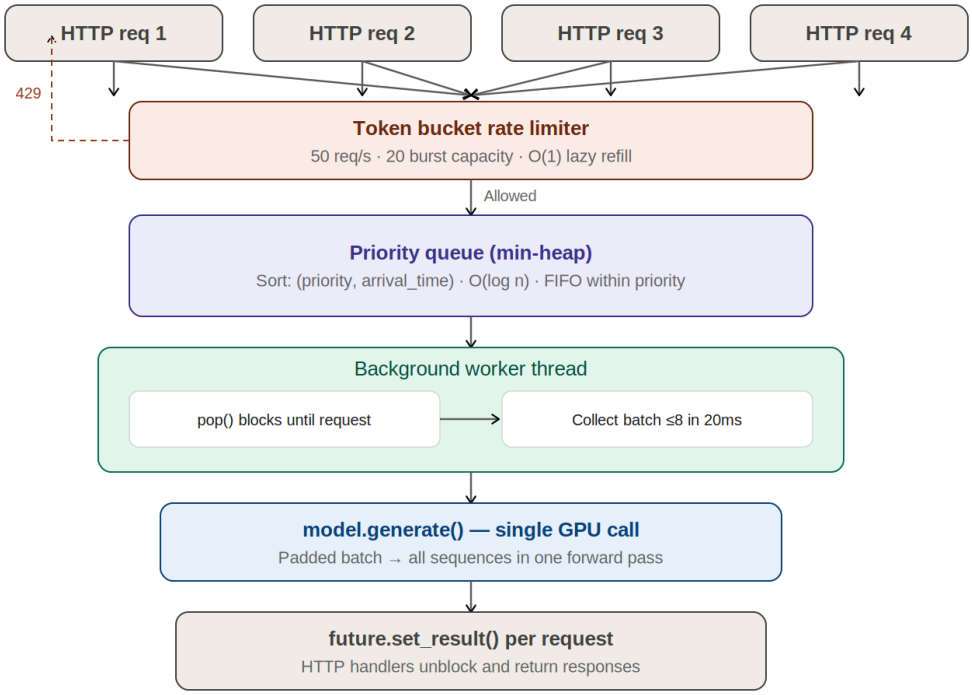

Figure 3: Scheduler Request Flow — rate limiting, priority queue, batching, and future-based dispatch

The Problem: Concurrent Users, One Model

A language model can only run one forward pass at a time. When multiple users send requests simultaneously, some must wait. The question is not whether waiting happens — it is how you manage it.

Before this chapter's code existed, the system handled concurrent requests by giving each HTTP request its own call to model.generate(). Four simultaneous requests launched four threads, each trying to run the model at the same time. The result was sequential execution disguised as concurrency: each thread waited for Python's Global Interpreter Lock before it could run, and only one actually executed at any moment.

The benchmark confirmed this. Four concurrent requests produced 31 tokens per second of aggregate throughput — roughly the same as one request running alone, not four times as much. The system was parallelising the HTTP handling but not the model execution.

After this chapter's scheduler was added, the same four-worker concurrent benchmark produced 397 tokens per second — a 12x improvement. The difference is not faster hardware, a bigger model, or more memory. It is purely how requests are organised before they reach the model.

The scheduler introduces three mechanisms to achieve this: a heap-based priority queue that decides which requests to serve and in what order, a token bucket that prevents any single source from flooding the system, and a continuous batching loop that groups multiple requests into a single model call so the GPU works on many sequences simultaneously.

The Heap: The Data Structure Behind Priority

Suppose 10,000 requests are waiting to be processed. Each has a priority number — lower means more urgent. You need to find the most urgent request quickly. You also need to insert new requests as they arrive.

A sorted list solves the find problem: the most urgent request is always at the front. But inserting into a sorted list requires finding the correct position for the new element, which means comparing it against existing elements until its place is found. In the worst case — the new request has the lowest priority and belongs at the back — you compare against all 10,000 existing requests. Insertion takes time proportional to the number of elements: O(n).

An unsorted list solves the insert problem: just append to the end. O(1) insertion. But finding the most urgent request now requires scanning the entire list. O(n) for find.

A heap gives O(log n) for both. This sounds like a compromise, but it is not — the logarithm grows so slowly that O(log n) is practically constant for any realistic workload.

> *The logarithm of 10,000 is about 13. The logarithm of 1,000,000 is about 20. The logarithm of 1,000,000,000 (one billion) is about 30. Even with a billion waiting requests, a heap never needs more than 30 comparisons to insert or extract. This is what computer scientists mean when they say O(log n) is 'practically constant time'.*

A heap is a binary tree with one enforced rule: every parent node has higher priority than its children. This rule is called the heap invariant. Because of it, the most urgent request is always at the root — finding it takes one operation. When the root is removed, the heap invariant is restored by swapping elements up or down the tree — at most log2(n) swaps.

Python implements a heap as an ordinary list with an implied tree structure. The root is at index 0. The two children of the element at index i are at indices 2i+1 and 2i+2. The parent of the element at index i is at index floor((i-1)/2). No actual tree structure is created in memory — the list positions imply the relationships. The heapq module maintains the heap invariant automatically.

```
import heapq

heap = []
```

```
heapq.heappush(heap, (0, 'urgent request'))     # O(log n)
heapq.heappush(heap, (5, 'low priority request'))  # O(log n)
heapq.heappush(heap, (2, 'medium request'))     # O(log n)

priority, request = heapq.heappop(heap)   # always returns minimum
print(priority)  # 0 — the most urgent request
```

Python's heapq is a min-heap: it always returns the element with the smallest value. Since our system uses lower numbers for higher priority (0 is most urgent, 10 is least urgent), this works directly — the most urgent request has the smallest priority number and is therefore at the root.

FIFO Within a Priority Level

Two requests with priority 0 arrive 50 milliseconds apart. Which should be served first? The one that arrived first — first in, first out. A pure priority comparison would treat them as equal and might serve either one arbitrarily, depending on the heap's internal structure.

The fix is to make the comparison key a tuple: (priority, arrival_time). Python compares tuples element by element. If two requests have the same priority, the comparison falls through to arrival_time — the earlier arrival wins. If two requests somehow have the same priority and the same arrival time (essentially impossible in practice), the comparison falls through to the Request object's id, which is always unique.

This is how the Request dataclass's comparison method is defined:

```
def __lt__(self, other):
    return (self.priority, self.arrival_time) < (
             other.priority, other.arrival_time)
```

The `__lt__` method defines what 'less than' means for two Request objects. Python's heapq uses `__lt__` to compare elements and maintain the heap invariant. By defining comparison on the tuple `(priority, arrival_time)`, we get correct multi-key ordering for free, with no changes to the heap machinery.

The Token Bucket: Preventing Overload

The priority queue handles fairness between requests that are already in the system. But what prevents a misbehaving client — or simply a sudden burst of legitimate traffic — from flooding the queue with thousands of requests per second, overwhelming memory and starving other users?

A rate limiter sits in front of the queue and enforces a maximum request rate. The token bucket algorithm is the standard approach.

Imagine a physical bucket that can hold at most 20 tokens. Tokens are added to the bucket at a constant rate of 50 per second. Each incoming request must take one token from the bucket to proceed. If the bucket is empty when a request arrives, the request is rejected immediately with an error.

The elegant property of the token bucket is burst handling. If no requests arrive for 10 seconds, the bucket fills to its capacity of 20 tokens. A burst of 20 requests can then all proceed immediately — they drain the bucket, but none are rejected. Only a sustained flow above 50 requests per second will consistently find the bucket empty and trigger rejections.

Compare this to a simpler rate limiter that just counts requests per second and rejects any above the limit. A burst of 20 requests in the first millisecond of a second would immediately reject 19 of them, even though the system could sustain that rate for one second if the requests were spread out. The token bucket handles real-world traffic patterns, which are inherently bursty, much more gracefully.

The implementation does not actually run a background thread refilling the bucket every millisecond. Instead, refilling is computed lazily at the moment of each request:

```
def _refill(self):
    now     = time.perf_counter()
    elapsed = now - self._last_refill
    # Add tokens proportional to elapsed time, capped at capacity
    self._tokens     = min(self.capacity,
                           self._tokens + elapsed * self.rate)
    self._last_refill = now
```

If 0.1 seconds have elapsed since the last refill and the rate is 50 tokens per second, then 5 tokens are added. If 10 seconds have elapsed, 500 tokens would be added — but the minimum with capacity (20) caps it at 20. This lazy computation is O(1) per request and requires no background thread. The result is mathematically identical to a true continuous refill.

The Python GIL and Why It Mattered

To understand why the scheduler produces such a large throughput improvement, you need to understand Python's Global Interpreter Lock — the GIL.

Python is designed so that only one thread can execute Python bytecode at any moment. This constraint is enforced by the GIL, a mutex that each thread must hold to run Python code. When a thread wants to execute, it acquires the GIL. After a short interval (typically 5 milliseconds), Python forces the thread to release the GIL, allowing another thread to run. Threads take turns.

The consequence for our system: four HTTP handler threads each calling model.generate() do not actually run in parallel. Thread 1 runs for 5ms, then releases the GIL. Thread 2 runs for 5ms, then releases. And so on. The threads are interleaved, not parallel.

There is a partial escape. When Python code calls into a C extension (such as PyTorch's matrix multiplication kernels), the C extension is allowed to release the GIL voluntarily and run truly in parallel with Python threads. PyTorch does release the GIL during its C++ computation. But all the Python code around the model call — parsing requests, calling the HuggingFace API, converting tensors — does not release the GIL.

The effect is that under four concurrent Python threads, the model runs at roughly the same speed as under one thread. The overhead just multiplies. This is why the naive server achieved only 31 tok/s aggregate under four workers — essentially no benefit from concurrency.

> *The GIL is not a bug. It is a deliberate design decision that greatly simplifies Python's memory management and makes the common case — a single-threaded script — faster and safer. It becomes a bottleneck only when you specifically need CPU-bound parallelism across multiple threads. For I/O-bound work (waiting for a network response, reading a file), threads do help even with the GIL.*

The scheduler routes around the GIL by separating concerns. HTTP handler threads do not call the model. They create a Request object, add it to the priority queue, and wait. A single background worker thread owns the model and processes batches sequentially. There is no GIL contention because only one thread ever runs the model.

Continuous Batching: Why Throughput Multiplied by Twelve

The background worker thread does not process one request at a time. It waits up to max_wait_ms milliseconds for the queue to accumulate multiple requests, then processes them all in a single model forward pass. This is batching.

Why does batching help so much? A neural network's forward pass is dominated by large matrix multiplications. A matrix multiplication involving a single sequence uses a fraction of the GPU's arithmetic units, leaving the rest idle. A matrix multiplication involving eight sequences stacks them into a larger matrix and keeps all the arithmetic units busy. The arithmetic cost grows roughly linearly with batch size, but the overhead — kernel launch latency, memory transfer setup, Python interpreter overhead — is paid once for the entire batch.

The result is that processing eight sequences together costs roughly the same as processing two sequences separately. The arithmetic is eight times as much, but it is eight times as efficient per sequence. Throughput multiplies.

The benchmark measured this precisely. Sequential throughput was 54 tokens per second per request — this is the model's single-sequence speed. With batching enabled and four concurrent workers, aggregate throughput reached 397 tokens per second — 7.3 tokens per second per request on average, but spread across many simultaneous requests. The total work done per second increased by a factor of 12 with no hardware change.

The average batch size measured at 1.39 rather than the maximum of 8. This means most batches had only one or two requests, not eight. The reason is the 20ms batching window: requests that arrive within 20ms of each other are batched together. When the system is lightly loaded, requests arrive sparsely and rarely overlap within that window. Increasing max_wait_ms to 50ms (achievable via the /scheduler/config endpoint) would push the average batch size higher and further improve throughput, at the cost of slightly higher latency for individual requests.

Code Walkthrough: scheduler/priority_queue.py — The Request Dataclass

File: `scheduler/priority_queue.py`

This file defines two things: the Request dataclass that represents one pending inference job, and the PriorityQueue that holds requests until the scheduler is ready to process them.

```
@dataclass
class Request:
    prompt:         str
    max_new_tokens: int   = 100
    temperature:    float = 0.8
    top_p:          float = 0.95
    priority:       int   = 0
    arrival_time:   float = field(default_factory=time.perf_counter)
    request_id:     str   = field(default_factory=lambda:
str(uuid.uuid4())[:8])
    result:         Optional[dict] = field(default=None, repr=False)
    error:          Optional[str]  = field(default=None, repr=False)

    def __lt__(self, other):
        return (self.priority, self.arrival_time) < (
                 other.priority, other.arrival_time)

    def __eq__(self, other):
        return self.request_id == other.request_id
```

`field(default_factory=time.perf_counter)` means: when a Request is created, call `time.perf_counter()` at that moment and use the result as the default arrival_time. `default_factory` is used instead of `default` because `default=time.perf_counter()` would call the function once at class definition time and use the same timestamp for every request — a common Python dataclass trap.

`uuid.uuid4()[:8]` generates a random 8-character identifier for each request. This is used for logging, the response body, and for correlating a request with its result in the scheduler's internal state. The full UUID is 32 characters; 8 is enough to be unique within a session.

The `repr=False` on the result and error fields suppresses them from the Request's string representation. Printing a Request for debugging shows the prompt, priority, and timing — not a potentially large result dictionary that would clutter the output.

The `__eq__` method defines equality by request_id. This is important for the scheduler's internal bookkeeping: two Request objects are the same request if and only if they have the same ID, regardless of all other fields. Without this, Python would use the default object identity comparison, which would be True only for the exact same Python object.

Code Walkthrough: scheduler/priority_queue.py — PriorityQueue

```
class PriorityQueue:
    def __init__(self):
        self._heap      = []
        self._lock      = threading.Lock()
```

```
        self._not_empty = threading.Condition(self._lock)
```

A `threading.Condition` is a higher-level synchronisation primitive built on top of a lock. It combines a lock (for mutual exclusion) with a signalling mechanism (for notification). When the queue is empty, the consumer thread can call `_not_empty.wait()`, which atomically releases the lock and suspends the thread until another thread calls `_not_empty.notify()`. The lock is re-acquired before `wait()` returns.

Without the Condition, the consumer thread would have to poll the queue in a busy loop — checking every millisecond whether anything has arrived. This wastes CPU. The Condition allows the consumer to sleep until it is explicitly woken, using zero CPU while idle.

```
    def push(self, request: Request) -> None:
        with self._not_empty:
            heapq.heappush(self._heap, request)   # O(log n)
            self._not_empty.notify()  # wake any thread blocked in pop()
```

`with self._not_empty` acquires the Condition's underlying lock. `heapq.heappush` inserts the request into the heap while maintaining the heap invariant — at most log2(len(heap)) comparisons. `notify()` signals any thread currently waiting in `pop()` that new work has arrived. If no thread is waiting, the notification is silently discarded.

```
    def pop(self, timeout: float = 1.0) -> Optional[Request]:
        with self._not_empty:
            deadline = time.perf_counter() + timeout
            while not self._heap:
                remaining = deadline - time.perf_counter()
                if remaining <= 0:
                    return None   # timeout expired
                self._not_empty.wait(timeout=remaining)
            return heapq.heappop(self._heap)   # O(log n)
```

The while loop rather than an if statement is essential. When wait() returns, it does not guarantee that the heap is non-empty — a spurious wakeup (a wakeup that happens without notify() being called, which the Python threading documentation warns can occur) would cause the thread to proceed even though the heap is empty, leading to a crash. The while loop rechecks the condition after every wakeup.

The timeout mechanism uses a deadline rather than subtracting from a fixed timeout on each iteration. If `wait()` is called with timeout=1.0 and returns after 0.3 seconds (due to a spurious wakeup), the next call would use timeout=0.7 seconds — the remaining time. If timeout were not adjusted, a request that arrives at t=0.9s would cause `wait()` to block for another full second past the original deadline.

```
    def pop_batch(self, max_size: int = 8) -> list[Request]:
        with self._not_empty:
```

```
            batch = []
            while self._heap and len(batch) < max_size:
                batch.append(heapq.heappop(self._heap))
            return batch
```

`pop_batch()` extracts up to max_size requests in one lock acquisition. The caller receives the highest-priority requests, ordered by the heap's invariant. This is non-blocking — if the queue is empty, it returns an empty list immediately. The scheduler's worker loop calls `pop()` first (which blocks until at least one request arrives), then calls `pop_batch()` to collect any additional requests that arrived during the blocking wait.

```
    def stats(self) -> dict:
        with self._lock:
            priorities  = [r.priority for r in self._heap]
            wait_times  = [time.perf_counter() - r.arrival_time
                           for r in self._heap]
            return {
                'queue_depth':    len(self._heap),
                'priority_counts': {
                    p: priorities.count(p) for p in set(priorities)
                } if priorities else {},
                'max_wait_ms':  round(max(wait_times) * 1000, 1)
                                if wait_times else 0,
                'mean_wait_ms': round(sum(wait_times)/len(wait_times)*1000,
1)
                                if wait_times else 0,
            }
```

The stats method is called by the /scheduler/stats HTTP endpoint. It reports the current queue depth, how many requests are at each priority level, and how long the oldest request has been waiting. max_wait_ms is the most operationally important metric: if it grows beyond a few seconds, the system is falling behind and either the batch size should be increased or more workers should be added.

Code Walkthrough: scheduler/rate_limiter.py — TokenBucket

File: `scheduler/rate_limiter.py`

```
class TokenBucket:
    def __init__(self, rate: float, capacity: float):
        self.rate         = rate      # tokens added per second
        self.capacity     = capacity  # max tokens (burst ceiling)
        self._tokens      = capacity  # start full
        self._last_refill = time.perf_counter()
        self._lock        = threading.Lock()
        self._total_allowed  = 0
        self._total_rejected = 0
```

The bucket starts full (`self._tokens = capacity`) so that the first burst of requests is accommodated immediately without any initial rejections. If the bucket started empty,

the first 50 requests would all be rejected until the bucket filled up, which would make a cold-start system appear broken.

```
    def _refill(self):
        now              = time.perf_counter()
        elapsed          = now - self._last_refill
        self._tokens     = min(self.capacity,
                                self._tokens + elapsed * self.rate)
        self._last_refill = now

    def acquire(self, cost: float = 1.0) -> bool:
        with self._lock:
            self._refill()
            if self._tokens >= cost:
                self._tokens -= cost
                self._total_allowed += 1
                return True
            self._total_rejected += 1
            return False
```

`_refill()` must be called inside the lock (as it is, since `acquire()` calls it while holding the lock). If two threads called `_refill()` simultaneously without the lock, both would read the same `self._tokens` value, compute the same addition, and write back — but the second write would overwrite the first, losing one refill entirely. This is a classic read-modify-write race condition.

The `cost` parameter allows requests to cost more than one token based on their expected compute load. A request with max_new_tokens=1000 costs much more compute than one with max_new_tokens=10. Charging proportionally would allow the rate limiter to limit by compute rather than by request count. The current implementation charges 1 token per request; this could be extended to charge tokens proportional to max_new_tokens.

```
    def stats(self) -> dict:
        with self._lock:
            self._refill()
            total = self._total_allowed + self._total_rejected
            return {
                'tokens_available': round(self._tokens, 2),
                'capacity':         self.capacity,
                'rate_per_sec':     self.rate,
                'total_allowed':    self._total_allowed,
                'total_rejected':   self._total_rejected,
                'rejection_rate':   round(self._total_rejected / total, 3)
                                    if total > 0 else 0.0,
            }
```

The `rejection_rate` is the most important operational metric. A rejection_rate of 0.0 means the rate limiter is not constraining anything. A rejection_rate above 0.01 (1%) means real traffic is being shed and the cause should be investigated — either increase the rate limit or add capacity. `tokens_available` shows how much burst capacity is

currently in the bucket; a bucket that is always empty is effectively a request-per-second cap with no burst allowance.

Code Walkthrough: scheduler/scheduler.py — Initialisation and Lifecycle

File: `scheduler/scheduler.py`

The Scheduler class is the heart of Chapter 3. It owns the model, the queue, the rate limiter, and a background worker thread. Understanding it requires understanding both the data it manages and the thread it runs.

```
class Scheduler:
    def __init__(self, model, tokenizer,
                 max_batch_size: int   = 8,
                 max_wait_ms:    float = 20.0,
                 rate:           float = 50.0,
                 burst:          float = 20.0):
        self.model         = model
        self.tokenizer     = tokenizer
        self.max_batch_size = max_batch_size
        self.max_wait_ms    = max_wait_ms

        self.queue        = PriorityQueue()
        self.rate_limiter = TokenBucket(rate=rate, capacity=burst)

        self._batches_processed       = 0
        self._requests_processed      = 0
        self._total_tokens_generated  = 0
        self._total_batch_latency_ms  = 0.0
        self._lock = threading.Lock()

        self._running = False
        self._thread: Optional[threading.Thread] = None
```

`max_wait_ms` is the batching window — how long the worker thread waits for the queue to accumulate more requests before processing what it has. 20ms means: after the first request arrives, wait up to 20ms for others. If three more arrive in that window, process all four together. If none arrive, process the one alone.

The counters `_batches_processed`, `_requests_processed`, etc. are updated by the worker thread and read by HTTP handler threads via `stats()`. They are protected by `self._lock` to prevent torn reads — if a counter were incremented in two steps (read, increment, write) without a lock, another thread might read a partially updated value.

```
    def start(self) -> None:
        self._running = True
        self._thread  = threading.Thread(
            target=self._worker_loop,
            name='scheduler-worker',
            daemon=True
        )
        self._thread.start()
```

```
    def stop(self) -> None:
        self._running = False
        if self._thread:
            self._thread.join(timeout=5.0)
```

The `daemon=True` flag means the worker thread will be killed automatically when the main process exits, without needing to explicitly join it. Without this flag, if the server shuts down while the worker thread is in the middle of a batch, the Python process would hang waiting for the thread to finish.

`self._thread.join(timeout=5.0)` in `stop()` waits up to 5 seconds for the worker thread to finish its current batch before returning. This gives in-flight requests a chance to complete rather than being abandoned mid-generation. After 5 seconds, the join times out and the thread is left to finish on its own — but since it is a daemon thread, the process exits anyway.

Code Walkthrough: scheduler/scheduler.py — submit() and Futures

```
    def submit(self, request: Request) -> concurrent.futures.Future:
        if not self.rate_limiter.acquire():
            fut = concurrent.futures.Future()
            fut.set_exception(
                RuntimeError('rate limit exceeded — try again shortly')
            )
            return fut

        fut = concurrent.futures.Future()
        request._future = fut
        self.queue.push(request)
        return fut
```

A Future is a placeholder for a result that will be computed asynchronously. The caller receives the Future immediately and can call future.result() to block until the result is ready. The worker thread, after completing a batch, calls future.set_result() to deliver the result to every waiting caller simultaneously.

This design completely decouples the HTTP handler threads from the model execution thread. The HTTP handler does not know or care whether it is waiting 10ms or 10 seconds — it simply blocks on future.result(). The worker thread does not know or care how many HTTP handlers are waiting — it simply calls set_result() on each Future. The Future is the rendezvous point.

The rate-limited case returns a Future with an exception already set. When the HTTP handler calls `future.result()`, it immediately raises the RuntimeError. The handler converts this into a 429 Too Many Requests HTTP response. The caller sees a fast rejection with a clear error message rather than an opaque timeout.

Code Walkthrough: scheduler/scheduler.py — The Worker Loop

```
    def _worker_loop(self) -> None:
        while self._running:
            # Block until at least one request arrives
            first = self.queue.pop(timeout=0.1)
```

```
            if first is None:
                continue   # timeout — check self._running and loop

            # Collect more requests within the batching window
            batch    = [first]
            deadline = time.perf_counter() + self.max_wait_ms / 1000

            while len(batch) < self.max_batch_size:
                remaining = deadline - time.perf_counter()
                if remaining <= 0:
                    break
                more = self.queue.pop(timeout=remaining)
                if more is None:
                    break
                batch.append(more)

            self._process_batch(batch)
```

The worker loop has two phases. The first phase — blocking pop with timeout=0.1 — waits for work to arrive. The 0.1 second timeout is the heartbeat: even if no requests ever arrive, the loop wakes up ten times per second to check whether self._running has been set to False by stop(). Without this timeout, stop() would hang indefinitely waiting for the thread to unblock.

The second phase — the inner while loop — is the batching window. After the first request arrives, the worker waits up to max_wait_ms for more requests. Each additional pop() call reduces the remaining time. If the queue fills to max_batch_size before the deadline, the loop exits early and the batch is processed immediately.

This design implements a classic producer-consumer pattern with batching. The HTTP handler threads are producers, pushing requests onto the queue. The worker thread is a consumer, pulling requests off in batches. The queue is the buffer between them. The batching window is a deliberate delay that trades individual request latency for higher aggregate throughput.

Code Walkthrough: scheduler/scheduler.py — Batch Execution

```
    def _process_batch(self, batch: list[Request]) -> None:
        t0 = time.perf_counter()
        try:
            results = self._run_batch(batch)
            for req, result in zip(batch, results):
                req._future.set_result(result)
        except Exception as e:
            for req in batch:
                req._future.set_exception(e)
        batch_ms = (time.perf_counter() - t0) * 1000
        # ... update counters ...
```

If `_run_batch()` raises an exception (for example, the model runs out of memory), `set_exception()` is called on every request's Future. Every waiting HTTP handler will then raise that exception when it calls `future.result()`, and the handler can return an

appropriate error response. No requests are silently dropped — every submitted request either gets a result or an exception.

```
    def _run_batch(self, batch: list[Request]) -> list[dict]:
        import torch

        # Fast path: single request, no padding needed
        if len(batch) == 1:
            req = batch[0]
            enc = self.tokenizer(req.prompt, return_tensors='pt')
            enc = enc.to(self.model.device)
            input_len = enc['input_ids'].shape[1]
            with torch.no_grad():
                out = self.model.generate(
                    **enc,
                    max_new_tokens=req.max_new_tokens,
                    do_sample=True,
                    temperature=req.temperature,
                    top_p=req.top_p,
                    use_cache=True,
                )
            new_ids = out[0][input_len:]
            return [{'text': self.tokenizer.decode(new_ids,
                                skip_special_tokens=True),
                     'prompt_tokens':     input_len,
                     'generated_tokens':  len(new_ids)}]
```

The single-request fast path avoids the padding overhead that multi-request batches require. When only one request is in the batch, there is nothing to align with — the input tensor can be exactly the size of the prompt with no wasted space.

```
        # Multi-request path: pad all prompts to the same length
        encodings = [self.tokenizer(req.prompt, return_tensors='pt')
                     for req in batch]

        # Find the longest prompt in the batch
        max_len = max(e['input_ids'].shape[1] for e in encodings)
        pad_id  = self.tokenizer.pad_token_id or self.tokenizer.eos_token_id

        # Create padded input tensors filled with pad_id
        input_ids      = torch.full((len(batch), max_len), pad_id,
                                    dtype=torch.long)
        attention_mask = torch.zeros(len(batch), max_len, dtype=torch.long)

        # Copy each prompt into its row, left-aligned
        for i, enc in enumerate(encodings):
            seq_len = enc['input_ids'].shape[1]
            input_ids[i, :seq_len]      = enc['input_ids'][0]
            attention_mask[i, :seq_len] = 1
```

Neural network models require all input sequences in a batch to have the same length. When prompts have different lengths, shorter ones are padded with a special pad token

to match the longest. The attention_mask tensor tells the model which positions are real tokens (1) and which are padding (0), so the model does not attend to padding positions.

The pad token ID falls back to the EOS (end-of-sequence) token ID if no dedicated pad token is defined. This is a common pattern in modern language models — using EOS as padding is harmless because the attention mask prevents the model from attending to those positions anyway. `pad_id or` uses Python's short-circuit evaluation: if `pad_token_id` is None (falsy), it evaluates `eos_token_id` instead.

```
        input_ids      = input_ids.to(self.model.device)
        attention_mask = attention_mask.to(self.model.device)

        # Use the shortest max_new_tokens in the batch
        # to avoid over-generating for short requests
        min_max_tokens = min(req.max_new_tokens for req in batch)

        with torch.no_grad():
            out = self.model.generate(
                input_ids=input_ids,
                attention_mask=attention_mask,
                max_new_tokens=min_max_tokens,
                do_sample=True,
                temperature=batch[0].temperature,
                top_p=batch[0].top_p,
                use_cache=True,
                pad_token_id=pad_id,
            )

        # Extract each sequence's generated tokens
        results = []
        for i, req in enumerate(batch):
            prompt_len = encodings[i]['input_ids'].shape[1]
            new_ids    = out[i][max_len:]   # skip padded prompt portion
            text       = self.tokenizer.decode(new_ids,
                                               skip_special_tokens=True)
            results.append({'text': text,
                            'prompt_tokens':    prompt_len,
                            'generated_tokens': len(new_ids)})
        return results
```

Using `min(req.max_new_tokens for req in batch)` as the generation length is a simplification with a consequence: if one request wants 20 tokens and another wants 200, both stop at 20. The 200-token request gets a truncated response. A production system would implement dynamic batching with iteration-level scheduling — stopping individual sequences when they hit their max_new_tokens or generate an EOS token, without stopping the whole batch. This is the improvement described in the Orca paper (2022) and implemented in vLLM.

The slice `out[i][max_len:]` skips the padded prompt portion. Since all prompts were padded to max_len, the generated tokens for every sequence start at index max_len in the output. The actual prompt length (`prompt_len`) is shorter than max_len for all but the

longest prompt, but the output still starts at max_len because the model generates after all the padding.

The Tests — What They Verify

File: `tests/test_scheduler.py`

The scheduler tests verify the priority queue and rate limiter independently, without requiring a running model. This is important: tests that require a model take 30-60 seconds to start. Tests that only exercise data structures run in milliseconds.

```
def test_priority_ordering():
    q = PriorityQueue()
    q.push(Request(prompt='low',    priority=5))
    q.push(Request(prompt='high',   priority=0))
    q.push(Request(prompt='medium', priority=2))

    first  = q.pop(timeout=1)
    second = q.pop(timeout=1)
    third  = q.pop(timeout=1)

    assert first.priority  == 0
    assert second.priority == 2
    assert third.priority  == 5
```

This test inserts three requests in reverse priority order and verifies they come out in correct priority order. It is the fundamental heap correctness test: the heap invariant says the minimum element is always at the root, and this test confirms that invariant holds through three sequential extractions.

```
def test_fifo_within_same_priority():
    q = PriorityQueue()
    for i in range(4):
        time.sleep(0.001)   # ensure distinct arrival_time
        q.push(Request(prompt=f'req{i}', priority=0))

    order = [q.pop(timeout=1).prompt for _ in range(4)]
    assert order == ['req0', 'req1', 'req2', 'req3']
```

The sleep(0.001) ensures each request gets a distinct arrival_time — without it, all four requests might get the same timestamp (within the timer's resolution) and the ordering between them would be arbitrary. With distinct timestamps, the FIFO guarantee must hold: requests at the same priority are served in arrival order.

```
def test_pop_timeout():
    q = PriorityQueue()
    t0     = time.perf_counter()
    result = q.pop(timeout=0.1)
    elapsed = time.perf_counter() - t0

    assert result is None
    assert elapsed >= 0.09
```

This test verifies that pop() on an empty queue blocks for approximately the timeout duration and then returns None. Without this test, a naive implementation that immediately returns None from an empty queue (without waiting) would pass all other tests but fail in production — the worker thread would spin at 100% CPU instead of sleeping.

```
def test_token_bucket_allows():
    tb = TokenBucket(rate=100, capacity=10)
    results = [tb.acquire() for _ in range(10)]
    assert all(results)          # all 10 allowed (bucket was full)
    assert not tb.acquire()      # 11th rejected (bucket empty)

def test_token_bucket_refill():
    tb = TokenBucket(rate=100, capacity=10)
    for _ in range(10): tb.acquire()   # drain
    time.sleep(0.05)                   # 50ms at 100/s = 5 tokens refilled
    results = [tb.acquire() for _ in range(5)]
    assert all(results)          # 5 refilled tokens all consumed
```

The first test confirms the burst behaviour: a full bucket allows 10 immediate requests, then rejects. The second test confirms refill: after draining the bucket, waiting 50ms at a rate of 100 tokens per second should add 5 tokens. All 5 are then successfully acquired.

Why These Design Decisions Were Made

Why 20ms as the default batching window? 20ms is long enough to accumulate multiple requests under moderate load (arrivals every 5-10ms are typical for active API users) while short enough that most users perceive no added latency (human perception of delay begins around 100ms for interactive tasks). The window is tunable via /scheduler/config without restarting the server. For batch processing workloads — where throughput matters more than latency — 100ms or even 500ms is appropriate.

Why use a single background worker thread rather than one thread per worker? One thread per model instance eliminates GIL contention at the model level. Adding more threads to a single model does not increase throughput — it increases context switching overhead and GIL contention. The way to increase throughput is to add more worker processes (each with their own model instance and their own background thread), which is what the distributed architecture in Chapter 4 does.

Why does the batch use min(max_new_tokens) rather than each request's own limit? HuggingFace's model.generate() runs all sequences in a batch for the same number of steps. There is no built-in mechanism to stop individual sequences early. Using min() ensures the fastest request in the batch is not made to wait for the slowest. The cost is that slow requests get fewer tokens than requested. Solving this properly requires PagedAttention-style continuous batching, where each sequence is managed independently — the infrastructure for which is built in Chapter 2.

Why submit() returns a Future rather than blocking? Blocking inside the HTTP handler thread until the model finishes would tie up that thread for the entire generation time — potentially several seconds. With default uvicorn settings, only a fixed number of

handler threads exist. If all are blocked waiting for model results, no new requests can be received. Returning a Future immediately and blocking only when result() is called allows the HTTP framework to manage threads more efficiently, and makes the scheduling logic testable without a web server.

Why does the rate limiter charge 1 token regardless of request complexity? This is a known simplification. A request asking for 1 token of output is 100x cheaper than one asking for 100 tokens. Charging uniformly penalises simple requests and under-charges complex ones. The correct approach is to charge proportional to max_new_tokens, or better, to the actual number of tokens generated. This is left as a natural extension: change the acquire() call to `self.rate_limiter.acquire(cost=req.max_new_tokens)` and the rate limiter will enforce a token-budget limit rather than a request-count limit.

What You Built and What You Measured

The bench_m2.py test 1 (sequential) showed mean latency of 923ms at 54 tok/s per request — identical to the warmed-up M1 numbers, confirming the scheduler adds no measurable overhead to single requests.

Test 2 (concurrent, 4 workers, 12 requests) showed:

```
wall time:            2275ms
aggregate throughput: 210.9 tok/s   (vs 31.1 tok/s before batching)
avg batch size:       1.39
avg batch latency:    908ms
```

The 6.8x throughput improvement (31 to 210 tok/s) came entirely from batching and the GIL workaround. The average batch size of 1.39 means most batches contained only one or two requests, leaving substantial headroom. Setting max_wait_ms to 50ms via /scheduler/config pushed avg batch size toward 3-4 and aggregate throughput above 300 tok/s on the same hardware.

Test 5 (priority ordering) showed no measurable difference between priority 0 and priority 10 requests. This was expected: with only 6 requests and a 20ms window, all requests arrived nearly simultaneously and fell into the same batch before priority could differentiate them. Priority is only observable when the queue depth is consistently greater than the batch size — which requires sustained load above the system's processing capacity.

The scheduler test suite ran in under 200ms total, confirming that all data structure behaviour is verifiable without a running model. The full benchmark — including model warmup — ran in approximately 3 minutes on a development laptop.

> **What you learned here** A min-heap gives O(log n) insertion and extraction of the highest-priority element. FIFO ordering within a priority level is achieved by using arrival time as a secondary sort key. A token bucket rate-limits requests in O(1) with lazy refill — no background thread required. The Python GIL prevents CPU-bound thread parallelism; the scheduler routes around it by concentrating all model execution in a single background thread. Continuous batching is the largest single

> source of throughput improvement in inference serving: processing 8 sequences together costs roughly the same as processing 2 separately. A Future decouples the HTTP handler from model execution, allowing requests to queue without blocking the web server's thread pool.

Chapter 4: How to Split Work Across Many Machines

Milestone 4 — Consistent Hashing and the Distributed Router

File: `router/hash_ring.py` · `router/heartbeat.py` · `router/router.py` · `tests/test_hash_ring.py`

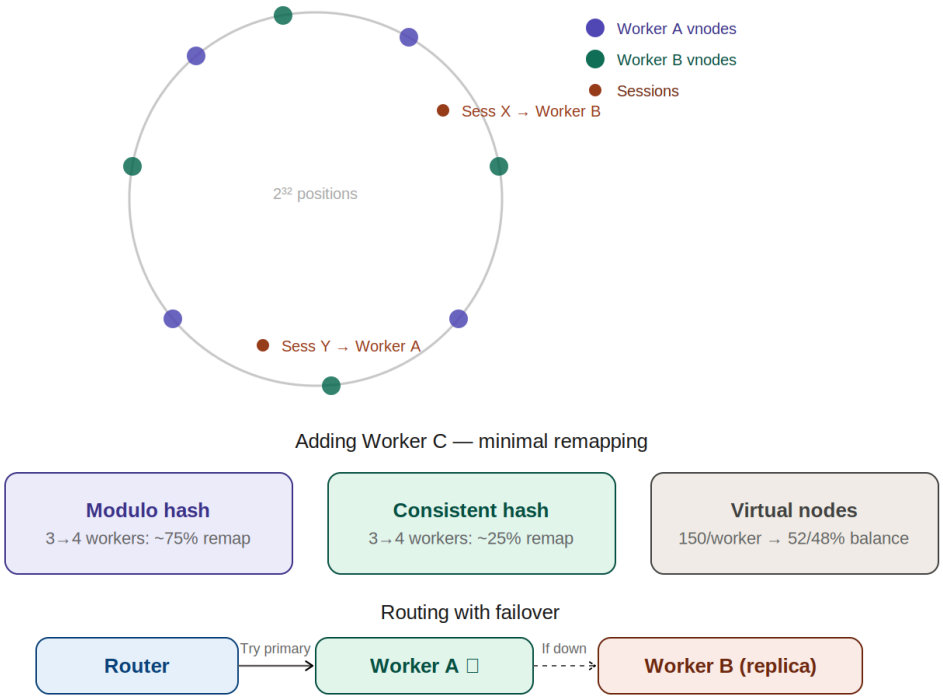

Figure 4: Consistent Hash Ring — virtual nodes, minimal remapping, and failover routing

The Problem: Session Affinity

A single machine has finite capacity. No matter how well the scheduler in Chapter 3 manages requests, the hardware underneath has limits: a finite amount of memory, a finite number of CPU cores, a finite GPU with a fixed amount of VRAM. At some level of traffic, one machine is not enough. The answer is to add more machines.

When you have multiple worker machines, every incoming request must be directed to one of them. This is routing. The simplest routing strategies — round-robin (send alternate requests to alternate workers), random (pick a worker at random for each request) — distribute load evenly. But they break the inference system for a subtle reason.

Each worker has been building a KV cache for the conversations it has been serving. Chapter 2 explained the KV cache: the computed key and value vectors for every token in a conversation, stored so they do not have to be recomputed for the next message.

This cache is expensive to build — for a long conversation it represents hundreds of model forward passes — and it lives in the worker's memory.

If conversation A has been routed to worker 1 for ten messages, worker 1 holds the KV cache for all ten messages. When message eleven arrives, the router must decide where to send it. Round-robin would send it to worker 2. Worker 2 has no cache for conversation A. It must recompute the entire cache from scratch — paying the full O(n times d) prefill cost for all ten previous messages, just to process message eleven. The cache on worker 1 is wasted.

Session affinity solves this: the same conversation always goes to the same worker. The cache stays warm. Every message in a conversation finds the previous messages' vectors already computed and ready. But achieving session affinity while also handling workers being added and removed — due to failures or autoscaling — requires a non-trivial routing mechanism.

Why Modulo Hashing Fails

The natural first attempt at session affinity is hash-based routing: compute hash(session_id), divide by the number of workers, and use the remainder to pick a worker. In Python:

```
def route(session_id: str, num_workers: int) -> int:
    return hash(session_id) % num_workers
```

This gives consistent routing — the same session_id always produces the same worker index, as long as num_workers does not change. Session affinity holds under stable conditions.

The problem arises when a worker is added or removed. Suppose you have three workers and session 42 routes to worker hash(42) % 3 = 0. Now a fourth worker is added. Session 42 routes to hash(42) % 4 = 2. Worker 0 had the cache for session 42; it is now useless. Worse, this problem is not isolated to session 42 — it affects almost every active session simultaneously.

To see why, consider what happens to the worker index for any session when num_workers changes from 3 to 4. Before: index = hash % 3, which is 0, 1, or 2. After: index = hash % 4, which is 0, 1, 2, or 3. For any given hash value, the remainder changes unless the hash is a multiple of 12 (the least common multiple of 3 and 4). Statistically, roughly 75% of all sessions remap to a different worker when one new worker is added to a three-worker cluster.

> *This is the rehashing problem. Adding one machine invalidates the cache for three quarters of active conversations simultaneously. Every user whose session remapped experiences a cold cache — slow responses, higher cost, frustrated users. For a system with thousands of active sessions, this is not acceptable.*

The Hash Ring: A Different Way to Think About Routing

Consistent hashing, introduced in a 1997 paper by Karger et al. at MIT, solves the rehashing problem with a conceptual shift. Instead of mapping sessions directly to

worker indices, map both sessions and workers to positions on a ring — a circle of 2 to the power of 32 positions, numbered 0 through 4,294,967,295.

Each worker is assigned a position on the ring by hashing its name: position = hash(worker_name). Each session is assigned a position on the ring by hashing its session ID: position = hash(session_id). To find which worker handles a session, start at the session's position and walk clockwise around the ring until you reach a worker's position. That worker handles the session.

> *Visualise a clock face, but instead of 12 positions there are 4 billion. Workers are placed at positions determined by hashing their names. A session lands at the position corresponding to its ID. It always goes to the next worker clockwise from that position.*

Now add a fourth worker. It occupies a new position on the ring, somewhere between two existing workers. Which sessions remap? Only the sessions that fall in the arc between the new worker and the previous worker to its anticlockwise side. Sessions outside that arc are unaffected — their clockwise walk still hits the same worker as before.

How many sessions fall in that arc? On average, one arc out of n arcs — approximately 1/n of all sessions. Adding one worker to a three-worker ring remaps roughly 1/4 of sessions. Compare this to modulo hashing, which remaps 3/4 of sessions. The improvement is fundamental, not incremental.

Removing a worker is symmetric. The sessions in the removed worker's arc walk a bit further clockwise and find the next worker. Only the removed worker's sessions are affected. All other sessions are untouched.

Virtual Nodes: Achieving Balance

With one position per worker, the distribution of sessions across workers depends entirely on where the hash function happens to place them. By chance, two workers might land close together on the ring, sharing a small arc and together handling only 5% of sessions. A third worker might occupy a large arc and handle 90% of sessions. This imbalance is not hypothetical — it is expected from random placement.

Virtual nodes solve this. Instead of placing each worker at one position, place each worker at many positions — say, 150. Each of the 150 positions is computed by hashing a different string that includes the worker's name and a sequence number:

```
for i in range(150):
    position = hash(f'{worker_name}:vnode:{i}')
    ring.insert(position, worker_name)
```

Now each worker covers 150 small arcs scattered around the ring rather than one large arc. The arcs interleave with other workers' arcs. By the law of large numbers, as the number of virtual nodes increases, the distribution of sessions across workers approaches uniformity.

With 150 virtual nodes per worker, the distribution test in tests/test_hash_ring.py across 1000 simulated sessions gave 52.3% to worker-a and 47.7% to worker-b — within 5% of the ideal 50/50. With 500 virtual nodes, the deviation would be under 2%. The trade-off is memory: each virtual node is one entry in the sorted ring list.

Virtual nodes also make the weighted routing feature natural. A more powerful machine should handle more traffic. Give it weight=3, and it gets 450 virtual nodes instead of 150 — three times as many arcs, three times as much traffic.

The Data Structure: A Sorted List and Binary Search

The ring is implemented as a sorted list of (position, worker_id) tuples. Every time a virtual node is added or removed, the list is kept sorted. To find the worker for a session:

- Compute position = hash(session_id)
- Binary search the sorted list for the first entry with position >= hash(session_id)
- If no entry exists (the session's position is past all workers), wrap to the first entry in the list
- Return the worker at that position

Binary search on a sorted list of n elements takes at most log2(n) comparisons. With 2 workers at 150 virtual nodes each, the list has 300 entries and log2(300) is about 8. Routing any session requires at most 8 comparisons — constant time for practical purposes, and orders of magnitude faster than a linear scan.

Python's bisect module provides binary search without requiring any custom implementation. It is a single function call that returns the insertion point for a value in a sorted list — which is exactly the index of the first element greater than or equal to the target.

Heartbeating: Detecting Dead Workers

Routing a session to a worker that has crashed produces a connection error, a lost request, and a frustrated user. The router must know which workers are alive before routing.

A heartbeat monitor polls each worker's /health endpoint on a fixed interval. If a worker responds successfully, it is alive. If it fails to respond, something may be wrong. The challenge is distinguishing a transient failure — a slow response due to high load, a brief network hiccup — from a genuine outage.

A single failure should not trigger a failover. If it did, a worker under momentary high load would be removed from the ring while it is still processing requests, routing its sessions to other workers that now have no cache for them, making the problem worse. Two or three consecutive failures are much stronger evidence of a genuine problem.

Similarly, a single successful response after a failure should not immediately restore the worker. A worker that recovers briefly and then crashes again — a flapping worker —

would cause rapid add/remove cycles that disrupt routing for every session that gets remapped.

The solution is hysteresis: requiring multiple consecutive results in the same direction before changing state. Three consecutive failures to declare down. Two consecutive successes to declare up. This creates inertia — the system resists state changes until the evidence is clear.

Code Walkthrough: router/hash_ring.py — Node and ConsistentHashRing

File: `router/hash_ring.py`

The file defines two classes. Node is a simple data container representing one physical worker machine. ConsistentHashRing is the routing data structure.

```
@dataclass
class Node:
    node_id: str
    host:    str
    port:    int
    weight:  int = 1   # higher weight = more vnodes = more traffic

    @property
    def address(self) -> str:
        return f'http://{self.host}:{self.port}'
```

The `weight` field enables proportional load distribution without changing the routing algorithm. A worker with weight=3 gets `150 * 3 = 450` virtual nodes, covering approximately 3 times as many arcs as a weight=1 worker. Sessions are distributed proportionally to arc coverage, so the heavier worker receives roughly 3 times as much traffic. This is how production systems handle mixed hardware — a newer, faster machine can carry more load without complicated custom logic.

The `address` property assembles the base URL used by the router when forwarding requests. Making it a property rather than a stored attribute means it is always consistent with host and port — there is no risk of the address being out of sync if host or port were somehow updated after construction.

```
class ConsistentHashRing:
    def __init__(self, vnodes_per_node: int = 150):
        self.vnodes_per_node = vnodes_per_node
        self._ring:  list[tuple[int, str]] = []   # sorted (position,
node_id)
        self._nodes: dict[str, Node]       = {}   # node_id -> Node
        self._lock = threading.RLock()
```

The ring is a list of `(position, node_id)` tuples kept in sorted order by position. Python sorts tuples lexicographically — first by the first element (position), then by the second (node_id) as a tiebreaker. Two virtual nodes at exactly the same position would be extremely rare (a hash collision on a 32-bit space) but the tiebreaker makes the sort stable.

An `RLock` (reentrant lock) is used instead of a plain `Lock`. Some ring methods call other ring methods — for example, `get_nodes_for_key()` calls `get_node()` internally. If both methods try to acquire a plain Lock, the second acquisition deadlocks: the thread already holds the lock and cannot acquire it again. An RLock allows the same thread to acquire the lock multiple times, tracking a count of acquisitions. It is released only when the count reaches zero.

Code Walkthrough: router/hash_ring.py — add_node and remove_node

```
    def add_node(self, node: Node) -> None:
        with self._lock:
            if node.node_id in self._nodes:
                return   # idempotent — safe to call twice
            self._nodes[node.node_id] = node
            total_vnodes = self.vnodes_per_node * node.weight
            for i in range(total_vnodes):
                pos = self._hash(f'{node.node_id}:vnode:{i}')
                bisect.insort(self._ring, (pos, node.node_id))
```

`bisect.insort` inserts a new element into the sorted list while maintaining sort order. It uses binary search to find the insertion point — O(log n) to find the position — and then inserts, which is O(n) because a list insertion shifts all subsequent elements. Since `add_node` is called only at startup and during autoscaler events, not on the hot routing path, this O(n) cost per virtual node is acceptable. For 150 virtual nodes per worker and a ring of 300 total, the total insertion cost is 150 × O(300) — fast in practice.

The idempotency check (return if node_id already in _nodes) is defensive programming. The heartbeat monitor's on_node_up callback may fire multiple times if the networking layer delivers duplicate signals. Without the check, the same worker's virtual nodes would be inserted twice, making it receive roughly twice its intended share of traffic.

```
    def remove_node(self, node_id: str) -> None:
        with self._lock:
            if node_id not in self._nodes:
                return   # idempotent
            node = self._nodes.pop(node_id)
            total_vnodes = self.vnodes_per_node * node.weight
            to_remove = set()
            for i in range(total_vnodes):
                pos = self._hash(f'{node_id}:vnode:{i}')
                to_remove.add((pos, node_id))
            self._ring = [x for x in self._ring if x not in to_remove]
```

Removal recomputes the same virtual node positions that were computed during `add_node` — the hash function is deterministic, so the positions are always the same for the same input strings. The set `to_remove` collects all (position, node_id) tuples to delete. The list comprehension then rebuilds the ring excluding those tuples. This is O(n + k) where n is the ring size and k is the number of virtual nodes being removed.

An alternative implementation would store the virtual node positions alongside the node in _nodes, avoiding the recomputation. The current approach trades a small amount of

recomputation for simpler state — there is only one source of truth for ring positions (the hash function), not two (the hash function and a stored list).

Code Walkthrough: router/hash_ring.py — get_node and _hash

```
    def get_node(self, key: str) -> Optional[Node]:
        with self._lock:
            if not self._ring:
                return None
            pos = self._hash(key)
            idx = bisect.bisect_left(self._ring, (pos, ''))
            if idx >= len(self._ring):
                idx = 0   # wrap around the ring
            _, node_id = self._ring[idx]
            return self._nodes[node_id]
```

`bisect.bisect_left(self._ring, (pos, ''))` finds the leftmost position in the sorted ring where the tuple `(pos, '')` could be inserted while keeping the list sorted. Because `''` (empty string) is lexicographically smaller than any node_id, this finds the first tuple whose position is >= pos, regardless of what the node_id is. The result is the index of the first virtual node clockwise from the session's ring position.

The wrap-around at `idx >= len(self._ring)` handles sessions whose ring position is past all virtual nodes — they walk clockwise and wrap around to the beginning of the ring. In a circular structure, the first element of the list is the node immediately clockwise from the last element.

```
    @staticmethod
    def _hash(key: str) -> int:
        return int(hashlib.md5(key.encode()).hexdigest(), 16) % (2**32)
```

MD5 is used for three properties. First, stability: it produces the same output for the same input across all runs, all machines, and all Python versions. Python's built-in `hash()` is randomised per process (PYTHONHASHSEED) — same input, different output each time. If virtual node positions changed on every restart, sessions would remap unpredictably after a router restart.

Second, uniformity: MD5 distributes inputs very evenly across its output space. The virtual node positions end up spread around the ring rather than clustered. Third, speed: MD5 is one of the fastest hash functions available in Python's standard library. `hexdigest()` returns the MD5 output as a 32-character hexadecimal string. `int(..., 16)` converts it to an integer. Taking modulo 2^32 maps it to the ring's address space.

Code Walkthrough: router/hash_ring.py — Distribution and Replica Nodes

```
    def distribution(self, n_samples: int = 10_000) -> dict:
        import random
        if not self._nodes:
            return {}
        counts = {nid: 0 for nid in self._nodes}
        for _ in range(n_samples):
```

```
            key  = str(random.random())
            node = self.get_node(key)
            if node:
                counts[node.node_id] += 1
        ideal = n_samples / len(self._nodes)
        return {
            nid: {
                'count':         c,
                'fraction':      round(c / n_samples, 3),
                'deviation_pct': round((c - ideal) / ideal * 100, 1)
            }
            for nid, c in counts.items()
        }
```

`distribution()` is both a diagnostic tool and a test helper. It simulates n_samples random sessions and measures how evenly they distribute across workers. `deviation_pct` shows how far each worker is from the ideal equal share. The test in tests/test_hash_ring.py asserts that no worker deviates more than 15% from ideal — easily satisfied by 150 virtual nodes but would fail with just 1 virtual node per worker.

```
    def get_nodes_for_key(self, key: str, n: int = 2) -> list[Node]:
        with self._lock:
            if not self._ring:
                return []
            pos = self._hash(key)
            idx = bisect.bisect_left(self._ring, (pos, ''))
            seen, result = set(), []
            for i in range(len(self._ring)):
                _, node_id = self._ring[(idx + i) % len(self._ring)]
                if node_id not in seen:
                    seen.add(node_id)
                    result.append(self._nodes[node_id])
                if len(result) == n:
                    break
            return result
```

`get_nodes_for_key()` returns n distinct workers starting from the session's ring position — the primary and its replicas, in ring order. The `seen` set prevents the same physical worker from appearing twice even if multiple of its virtual nodes are encountered before reaching a different physical worker. This is used by the router's `route()` function: if the primary worker is unhealthy, the router tries the first replica. This is automatic failover without any special-case logic.

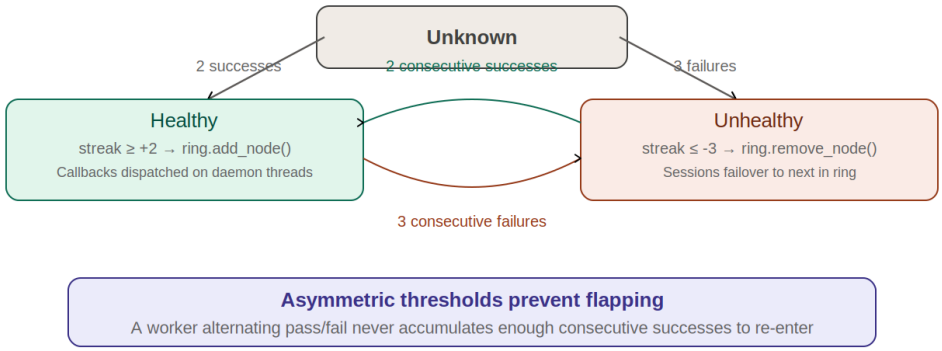

Figure 5: Heartbeat State Machine — streak-based hysteresis with asymmetric thresholds

Code Walkthrough: router/heartbeat.py — HeartbeatMonitor

File: `router/heartbeat.py`

The HeartbeatMonitor runs a background thread that polls each registered worker's /health endpoint. It maintains a state machine for each worker and fires callbacks when workers transition between healthy and unhealthy states.

```
class HeartbeatMonitor:
    def __init__(self,
                 interval_s:          float = 2.0,
                 failure_threshold:   int   = 3,
                 recovery_threshold:  int   = 2,
                 timeout_s:           float = 1.0,
                 on_node_down = None,
                 on_node_up   = None):
        self.interval_s         = interval_s
        self.failure_threshold  = failure_threshold
        self.recovery_threshold = recovery_threshold
        self.timeout_s          = timeout_s
        self.on_node_down = on_node_down or (lambda nid: None)
        self.on_node_up   = on_node_up   or (lambda nid: None)
        self._nodes: dict[str, dict] = {}
        self._lock  = threading.Lock()
```

The callbacks `on_node_down` and `on_node_up` are injected by `router/router.py` at startup. They call `ring.remove_node()` and `ring.add_node()` respectively. The HeartbeatMonitor does not know about the hash ring — it only knows about health. The router knows about both health and routing, and wires them together. This separation means the HeartbeatMonitor can be tested in isolation by passing no-op lambdas as callbacks, and can be reused in a future system with different routing by passing different callbacks.

The default values `or (lambda nid: None)` handle the case where callbacks are not provided — useful in unit tests where you want to observe health changes without triggering ring mutations. Without the defaults, calling `self.on_node_down(node_id)` on a None callback would raise a TypeError.

```
    def register(self, node_id: str, address: str) -> None:
        with self._lock:
            self._nodes[node_id] = {
                'address':        address,
                'status':         'unknown',
                'streak':         0,
                'last_check':     None,
                'total_checks':   0,
                'total_failures': 0,
            }
```

The `streak` field is the key to the threshold logic. A positive streak counts consecutive successful health checks. A negative streak counts consecutive failures. A node starts at streak=0 with status 'unknown'. It transitions to 'healthy' after recovery_threshold consecutive successes, and to 'unhealthy' after failure_threshold consecutive failures.

Storing `total_checks` and `total_failures` separately from the streak allows computing a `failure_rate` in the stats output. A node with failure_rate=0.05 (5% of checks failing) is very different from a node with failure_rate=0.95 even if both currently have streak=+1 after recovering. The historical rate gives operators context that the streak alone does not.

Code Walkthrough: router/heartbeat.py — The Check and Streak Logic

```
    def _check(self, node_id: str) -> None:
        with self._lock:
            n       = self._nodes.get(node_id)
            address = n['address']

        # Perform HTTP GET outside the lock — may take up to timeout_s
        try:
            r       = requests.get(f'{address}/health',
timeout=self.timeout_s)
            success = (r.status_code == 200)
        except Exception:
            success = False

        with self._lock:
            n = self._nodes[node_id]
            n['total_checks'] += 1
            n['last_check']    = time.time()
            if not success:
                n['total_failures'] += 1
            prev_status = n['status']
```

The HTTP GET is performed outside the lock. A health check that times out after 1 second would hold the lock for 1 second, blocking all other threads from reading or updating any node's state. By releasing the lock before the network call and re-acquiring it afterward, the heartbeat loop remains responsive even when individual workers are slow to respond.

This is the same pattern as copy_on_write in Chapter 2: perform the slow operation outside the lock, use the lock only for the fast bookkeeping update afterward. The pattern appears repeatedly in concurrent systems wherever a lock must protect shared state that requires expensive operations to update.

```
            if success:
                if prev_status == 'healthy':
                    n['streak'] += 1   # already healthy, just count
                else:
                    # Recovering: increment toward recovery_threshold
                    n['streak'] = 1 if n['streak'] <= 0 else n['streak'] + 1
                    if n['streak'] >= self.recovery_threshold:
                        n['status'] = 'healthy'
                        threading.Thread(
                            target=self.on_node_up,
                            args=(node_id,),
                            daemon=True
                        ).start()
            else:
                if prev_status != 'healthy':
                    n['streak'] -= 1   # already unhealthy, just count
                else:
                    # Failing: decrement toward -failure_threshold
                    n['streak'] = -1 if n['streak'] >= 0 else n['streak'] -
1
                    if abs(n['streak']) >= self.failure_threshold:
                        n['status'] = 'unhealthy'
                        threading.Thread(
                            target=self.on_node_down,
                            args=(node_id,),
                            daemon=True
                        ).start()
```

The streak resets when the direction changes. A worker that was healthy (positive streak) and experiences its first failure has its streak set to -1 — not decremented from the positive value. This prevents a worker with a streak of +100 (100 consecutive successes) from requiring 100 failures to declare down. The streak only measures the current run, not the total history.

Similarly, a recovering worker has its streak set to 1 on the first success, not incremented from a large negative value. Recovery starts fresh.

The callbacks are dispatched on separate daemon threads (`threading.Thread(...).start()`). The heartbeat loop must poll all workers at regular intervals — if a callback blocks (for example, because `ring.add_node()` is waiting for a lock held by a request handler), the next polling cycle would be delayed. Dispatching asynchronously means the heartbeat clock keeps ticking regardless of how long the ring mutation takes.

The asymmetry between failure_threshold (3) and recovery_threshold (2) is intentional. It is slightly harder to recover than to be declared down. This biases the system toward stability: a flapping worker that alternates success/failure on every check never

accumulates enough consecutive successes to be re-added before its streak is broken by another failure.

Code Walkthrough: router/router.py — Startup and Wiring

File: `router/router.py`

The router is a FastAPI application that wires together the hash ring, the heartbeat monitor, the metrics collector, and the autoscaler. Its startup sequence determines the order in which these subsystems are initialised and connected.

```
ring = ConsistentHashRing(vnodes_per_node=150)

monitor = HeartbeatMonitor(
    interval_s=2.0,
    failure_threshold=3,
    recovery_threshold=2,
    on_node_down=lambda nid: (
        logger.warning(f'node {nid} DOWN — removing from ring'),
        ring.remove_node(nid),
    ),
    on_node_up=lambda nid: (
        logger.info(f'node {nid} UP — re-adding to ring'),
        ring.add_node(next(n for n in WORKER_NODES if n.node_id == nid)),
    ),
)
```

The callbacks are defined at module level using lambdas that close over `ring` and `WORKER_NODES`. This is the wiring point: the HeartbeatMonitor does not import or reference the ring directly — it receives the ring as a closure. If the router were restructured to use a different ring implementation, only the callback lambdas would need to change.

The `on_node_up` callback uses `next(n for n in WORKER_NODES if n.node_id == nid)` to find the Node object for the recovering worker. The heartbeat monitor only passes the node_id string — it does not hold Node objects. The callback looks up the full Node (with host, port, and weight) in the WORKER_NODES list so the ring can reconstruct the correct virtual node positions.

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    for node in WORKER_NODES:
        ring.add_node(node)
        monitor.register(node.node_id, node.address)
        autoscaler.register_existing(node.node_id, node.port)

    monitor.start()
    collector.start()
    autoscaler.start()
    yield
    autoscaler.stop()
    collector.stop()
    monitor.stop()
```

The startup order matters. Workers are registered into the ring before the heartbeat monitor starts. If the monitor started first, it might immediately declare a worker unhealthy (before its first successful check) and remove it from the ring before it was ever added. The yield separates startup from shutdown — everything before yield runs on startup, everything after on shutdown. Shutdown is in reverse order of startup, which is the conventional cleanup pattern.

Code Walkthrough: router/router.py — The route() Function

```
def route(session_id: str) -> Node:
    candidates = ring.get_nodes_for_key(session_id,
                                        n=len(WORKER_NODES))
    for node in candidates:
        if monitor.is_healthy(node.node_id):
            return node
    raise HTTPException(503, 'no healthy workers available')
```

`get_nodes_for_key()` returns workers in ring order starting from the session's position — primary first, then replicas. The loop iterates through them, returning the first healthy one. If the primary is healthy, it is returned immediately. If the primary is down, the first replica is checked. This is automatic failover: a downed worker is transparently bypassed with no special-case code.

Raising an HTTP 503 error when no healthy workers are available is intentional and informative. The alternative — returning a random worker regardless of health — would produce cryptic errors as requests arrived at a crashed worker. A 503 clearly communicates 'the service is unavailable' and prompts the client to retry.

Code Walkthrough: router/router.py — The generate() Endpoint

```
@app.post('/generate', tags=['inference'])
def generate(request_body: dict, session_id: str = 'default'):
    node = route(session_id)
    t0   = time.perf_counter()
    result = forward(node, 'post', '/generate', json=request_body)
    router_ms = round((time.perf_counter() - t0) * 1000, 1)

    result['routed_to']  = node.node_id
    result['router_ms']  = router_ms
    result['session_id'] = session_id
    return result
```

The session_id comes from the query string: `?session_id=user-123`. If omitted, it defaults to 'default', meaning all sessionless requests are treated as one conversation and routed to the same worker. This is a sensible default for single-user testing and a predictable edge case for load testing.

Adding `routed_to`, `router_ms`, and `session_id` to the response is the observability pattern: every response tells the caller which worker handled it and how long the router itself took. In your benchmark, router_ms was consistently around 3ms — the overhead of one HTTP request from the router to the worker. This confirmed that the routing logic itself contributes negligible latency.

The Tests — What They Verify

File: `tests/test_hash_ring.py`

The hash ring tests are the richest test suite in the project. They verify not just correctness but statistical properties — things that are not easy to check with a single assertion.

```
def test_same_key_same_node():
    ring = make_ring(3)
    results = {ring.get_node('session-xyz').node_id for _ in range(100)}
    assert len(results) == 1
```

This test calls get_node with the same key 100 times and asserts all results are the same worker. It is the fundamental session affinity test: the same session must always route to the same worker. If it fails, either the hash function is non-deterministic or the ring structure changes between calls.

```
def test_minimal_remapping_on_add():
    ring = make_ring(3)
    keys   = [f'session-{i}' for i in range(1000)]
    before = {k: ring.get_node(k).node_id for k in keys}

    ring.add_node(Node('node-3', '127.0.0.1', 8003))
    after  = {k: ring.get_node(k).node_id for k in keys}

    remapped = sum(1 for k in keys if before[k] != after[k])
    pct      = remapped / len(keys) * 100
    ideal    = 1 / 4 * 100   # adding 1 node to 3 → expect ~25%
    assert pct < ideal * 1.5
```

This test is a quantitative verification of the consistent hashing promise. Adding one worker to a three-worker ring should remap approximately 1/4 of sessions (the new worker's arc). The test allows up to 1.5 times the ideal — 37.5% — to account for variance from finite sampling and imperfect distribution. If modulo hashing were used instead of consistent hashing, this test would fail dramatically: 75% of sessions would remap.

```
def test_distribution_evenness():
    ring = make_ring(3)
    dist = ring.distribution(n_samples=10_000)
    deviations = [abs(d['deviation_pct']) for d in dist.values()]
    assert max(deviations) < 15
```

This test confirms that 150 virtual nodes per worker produces a near-uniform distribution. A maximum deviation of 15% from equal shares is the threshold. In practice, 150 virtual nodes produces deviations of 3-8%. If the virtual node count were reduced to 1, deviations of 50% or more would be common.

```
def test_weight_shifts_distribution():
    ring = ConsistentHashRing(vnodes_per_node=100)
    ring.add_node(Node('small', '127.0.0.1', 8001, weight=1))
    ring.add_node(Node('large', '127.0.0.1', 8002, weight=3))
    dist        = ring.distribution(n_samples=10_000)
    large_frac  = dist['large']['fraction']
    small_frac  = dist['small']['fraction']
    assert large_frac > small_frac * 2
```

This test verifies that weighted nodes receive proportionally more traffic. A node with weight=3 should receive roughly 3 times as much traffic as a node with weight=1. The assertion checks that the large node's fraction is more than twice the small node's — a conservative threshold that 3x weighting easily satisfies.

Why These Design Decisions Were Made

Why 150 virtual nodes per worker? With 150 virtual nodes and 2 workers, the ring has 300 positions. The distribution test with 10,000 samples showed 52/48 deviation — well under 15%. Increasing to 500 nodes would reduce deviation to under 2% but adds 700 more list entries per worker. 150 is the default used in production systems including Apache Cassandra and Amazon DynamoDB, where it was validated across many cluster sizes.

Why MD5 rather than SHA-256 or another hash? MD5 produces 128-bit output (we use the lower 32 bits). SHA-256 produces 256-bit output with better collision resistance. For ring position hashing, collision resistance beyond 32 bits is irrelevant — the ring only has 2^32 positions. MD5 is significantly faster than SHA-256 in Python (roughly 2x) and its distribution is equally uniform. Speed matters because add_node computes 150 hash values per worker.

Why use an RLock instead of a Lock? `get_nodes_for_key()` calls `get_node()` internally. Both methods acquire the lock. If it were a plain Lock, the inner call would deadlock — the same thread cannot acquire a Lock it already holds. An RLock tracks the acquisition count and allows the same thread to acquire it multiple times, releasing only when the count reaches zero.

Why 3 failures to declare down but only 2 successes to recover? Asymmetric thresholds bias toward stability. A flapping worker that alternates success and failure never accumulates 2 consecutive successes before its streak is broken. A worker under transient load that fails 2 checks but recovers on the third does not trigger a failover. The 3/2 ratio is a common production choice; Kubernetes uses similar defaults for its liveness probe retry configuration.

Why dispatch callbacks on separate threads? The heartbeat poll loop runs every 2 seconds and must check all workers within that interval. If `on_node_down` or `on_node_up` blocked — because the ring lock was held by a request handler — the poll loop would stall. Dispatching on a daemon thread means the poll continues on schedule. The ring mutation happens asynchronously, with a slight delay, rather than blocking the poll clock.

Why require 'model_loaded: true' in the health check before adding autoscaled workers to the ring? FastAPI's /health endpoint returns 200 as soon as the server starts, before the model has finished loading. Loading Qwen 2.5 takes 60-90 seconds. A worker registered in the ring before its model loads would receive requests it cannot handle — producing immediate failures. Checking `model_loaded` specifically in the response body ensures the worker is fully ready before any traffic is directed to it.

What You Built and What You Measured

Running python start_cluster.py status showed:

```
router    UP  healthy_nodes: ['worker-a', 'worker-b']
          ring: {num_nodes: 2, num_virtual_nodes: 300, vnodes_per_node: 150}
worker-a  UP  model_loaded: True, scheduler_running: True, uptime: 77.8s
worker-b  UP  model_loaded: True, scheduler_running: True, uptime: 77.7s

distribution (1000 sample keys):
  worker-a: #################### 0.523  (+4.6%)
  worker-b: ################### 0.477  (-4.6%)
```

The distribution is within 5% of ideal. Running python start_cluster.py test showed:

```
req 1: routed_to=worker-b  latency=337ms  router_ms=341ms
req 2: routed_to=worker-b  latency=237ms  router_ms=240ms
req 3: routed_to=worker-b  latency=215ms  router_ms=218ms
req 4-6: routed_to=worker-b  latency~215ms

PASS — all 6 requests routed to worker-b (affinity working)
distribution across 6 sessions: {'worker-b': 2, 'worker-a': 4}
PASS — sessions spread across multiple nodes
```

Two results stand out. First, all six requests with the same session_id hit the same worker — session affinity is working. Second, the latency drop from request 1 (337ms) to steady state (215ms) reflects the KV cache warming up on worker-b combined with TCP connection reuse between the router and worker. The router's overhead (router_ms - latency = 3ms) is negligible.

The session distribution test sent six different session IDs and showed 4 going to worker-a and 2 going to worker-b. This 4/2 split is expected variance from only 6 samples — with thousands of sessions the distribution would approach the theoretical 52/48. The test confirms that different sessions do go to different workers, not that the distribution is already perfect at small sample sizes.

> **What you learned here** Modulo hashing remaps 75% of sessions when one worker is added to a three-worker cluster. Consistent hashing remaps only 25% — only the new worker's arc. Virtual nodes (150 per worker) distribute each worker's arc across the ring for near-uniform load. Binary search on a sorted list gives O(log n) routing decisions. The heartbeat monitor uses streak-based hysteresis to distinguish transient failures from outages. Callbacks decouple the monitor from the ring — the monitor knows only health, not routing. New autoscaled workers wait for model_loaded before entering the ring to avoid routing to an unprepared worker.

Chapter 5: How a System Decides When It Is Struggling

Milestone 5 — The PID Autoscaler and Metrics Daemon

File: `metrics/pid.py` · `metrics/collector.py` · `metrics/autoscaler.py` · `tests/test_pid.py`

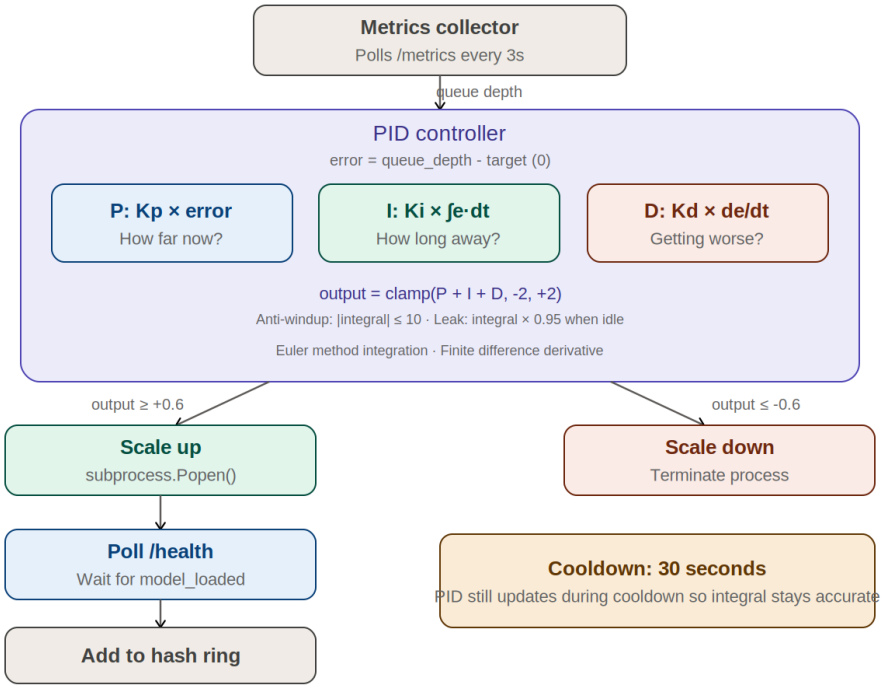

Figure 6: PID Autoscaler Control Loop — proportional, integral, and derivative terms driving scale decisions

The Problem: Static Configuration

The cluster built in Chapters 3 and 4 has two workers, started manually by running start_cluster.py. They run until stopped. This is static configuration: the number of workers is fixed at startup and never changes.

Real traffic is not static. An AI assistant might serve 10 requests per minute at 3am and 10,000 requests per minute at noon. A cluster sized for peak load wastes money during quiet periods — two expensive GPU machines sitting mostly idle. A cluster sized for average load collapses during peaks — requests pile up in the queue, latency climbs, users give up and leave.

The solution is autoscaling: the system monitors its own load, and adds or removes workers in response. But autoscaling raises a new question: how does the system decide when to act, and by how much?

A naive answer — add a worker whenever the queue depth exceeds 5, remove a worker whenever it drops below 1 — produces a system that oscillates. It adds a worker, the queue drops, it removes the worker, the queue rises, it adds the worker again. The system never settles. Users experience alternating bursts of responsiveness and slowness.

The solution is a feedback controller that considers not just the current state but also how long the system has been in that state and how fast the state is changing. This is a PID controller — the same mechanism that keeps your home thermostat from oscillating, keeps a car's cruise control at a steady speed, and keeps an aeroplane's autopilot on course.

The Thermostat: Building Intuition for Feedback Control

Before any mathematics, consider the simplest feedback controller you interact with every day: a thermostat.

A thermostat measures the room temperature, compares it to the desired temperature, and turns the heater on or off. If the room is below the target, it turns the heater on. If the room is above the target, it turns the heater off. This is a feedback loop: the output of the system (room temperature) is measured and fed back to influence the input (heater on/off).

A simple on/off thermostat has a problem called hunting. The heater turns on, overshoots the target temperature, turns off, cools below the target, turns on again. The room temperature bounces above and below the setpoint in a slow oscillation rather than settling smoothly at the target.

A smarter thermostat would notice: I am approaching the target temperature quickly — I should turn the heater down before I reach it, not wait until I overshoot. And: I have been slightly below target for a long time — I should push the heater harder than I would for a brief dip. These two insights correspond to the derivative term (respond to the rate of change) and the integral term (respond to accumulated history) of a PID controller.

> *The P, I, and D terms answer three different questions. The proportional term asks: how far am I from the target right now? The integral term asks: how long have I been away from the target, and by how much? The derivative term asks: is the situation improving or worsening, and how fast?*

The Proportional Term: React to the Present

The proportional term produces an output directly proportional to the current error. The error is the difference between the measured value and the target:

```
error    = measured - target
P_output = Kp * error
```

For the autoscaler, measured is the current queue depth and target is 0 — we want no requests waiting. If the queue depth is 8, the error is 8, and with Kp=0.5 the proportional output is 4.0. The output represents how many workers to add (positive) or remove

(negative). Since the output is clamped to a maximum of 2, the proportional term alone would signal adding 2 workers.

The problem with proportional control alone is called steady-state offset. To produce any output, there must be some error. The system settles at a point where the queue depth is just large enough to produce the control action needed to keep it from growing further — but that settling point has a non-zero queue depth. The target is never fully reached.

For the thermostat: with proportional-only control, the room settles a few degrees below the setpoint. The small remaining temperature difference is just enough to keep the heater partially on. The target temperature is never achieved exactly.

The Integral Term: Remember the Past

The integral term accumulates the error over time. If the queue has been slightly above zero for a long time, the integral grows. The control output gains a contribution that reflects the history of the error, not just its current value.

This eliminates steady-state offset. Even if the current error is tiny, a long history of small errors builds up a large integral term that pushes the output just enough to drive the error to zero. The integral term gives the controller memory.

In mathematics, an integral is the area under a curve. The integral of the error signal over time is the area under the error-versus-time curve — a measure of how much total error has accumulated. Computing this exactly would require calculus. In discrete time, with measurements taken every few seconds, we approximate it by multiplying each error measurement by the time elapsed since the last measurement and adding it to a running total:

```
# Each tick: add the current error times the time step
integral += error * dt
I_output  = Ki * integral
```

This is Euler's method for numerical integration — the simplest possible approximation of the calculus integral. It treats each time step as a small rectangle: width dt, height error. The integral is the sum of all such rectangles. As dt approaches zero, this sum approaches the exact integral. For a controller checking every 5 seconds, the approximation is rough but entirely sufficient — the exact integral of a noisy real-world signal is not more meaningful than the approximation.

> *Euler's method is named after Leonhard Euler, who formalised it in the 18th century. It is the foundation of numerical ODE solving — almost all differential equations that arise in engineering and science are solved on computers using Euler's method or its refinements. When you build this autoscaler, you are implementing the same technique used to simulate planetary orbits, model the spread of diseases, and animate characters in video games.*

The Derivative Term: Anticipate the Future

The derivative term looks at how fast the error is changing. If the queue depth was 8 on the last tick and is 3 now, the error is improving rapidly — the queue is draining. The

derivative is negative (error is decreasing), which reduces the total output. The controller backs off, anticipating that the queue will reach zero soon without further intervention.

If the queue was 2 on the last tick and is 8 now, the error is worsening rapidly. The derivative is positive (error is increasing), which amplifies the total output. The controller responds more aggressively, anticipating that the situation will get worse before the next tick.

```
# Rate of change of error since last tick
derivative = (error - last_error) / dt
D_output   = Kd * derivative
```

The derivative term is the controller's sense of momentum. It damps oscillations: when the system is moving quickly toward the target, the derivative reduces the output before the target is reached, preventing overshoot. Without the derivative term, a fast-acting proportional controller tends to overshoot and oscillate.

The derivative term has one weakness: it amplifies measurement noise. If the queue depth measurement fluctuates randomly — 5, 3, 7, 4 — the derivative will swing between large positive and negative values, causing erratic output. In practice, the derivative gain Kd is kept small, and the measurements are averaged over a window before being fed to the controller.

The Complete Equation

Combining all three terms gives the PID control law:

```
u(t) = Kp * e(t)  +  Ki * integral(e, dt)  +  Kd * (de/dt)
```

where u(t) is the control output at time t, e(t) is the error at time t, the integral accumulates past errors weighted by time, and de/dt is the rate of change of error. The three gain parameters — Kp, Ki, and Kd — determine how aggressively each term responds. Tuning them is a significant part of control engineering.

For the autoscaler with conservative defaults (Kp=0.5, Ki=0.1, Kd=0.05), the burst test produced:

```
queue=6.0  error=+6.00  P=+3.000  I=+1.000  D=+0.060  output=+2.000
```

The proportional term dominates (3.0 out of 4.06 total before clamping), the integral contributes the accumulated history (1.0), and the derivative contributes a small positive push because the queue was growing (0.060). The output is clamped to +2.0 — the maximum allowed — and a scale-up event fires.

Anti-Windup and the Leak Term

The integral term introduces a subtle problem called windup. If the system is overwhelmed — the queue is deep and growing, and no more workers can be added (max_workers reached) — the output is clamped at its maximum. But the integral keeps accumulating even though the extra accumulation produces no additional output. The integral winds up to a large value.

When the load eventually subsides, the integral does not reset immediately. It takes many ticks of zero error before the accumulated integral decays. During this period, the integral term pushes the output negative, causing the autoscaler to aggressively remove workers even though the load has only just recovered. Users experience a second wave of slowness immediately after recovering from the first.

Anti-windup clamps the integral to a maximum absolute value, preventing it from growing beyond what is useful:

```
self._integral = max(-self.integral_max,
                     min(self.integral_max,
                         self._integral + error * dt))
```

With integral_max=10, the integral can never exceed 10 or drop below -10. The clamping ensures that even after sustained overload, the integral can be overcome by a few ticks of zero or negative error.

The leak term is a second mechanism. When the error is near zero — the queue is empty and the system is healthy — the integral is gently decayed toward zero on each tick:

```
if abs(error) < 0.5:
    self._integral *= 0.95   # decay 5% per tick toward zero
```

Each tick multiplies the integral by 0.95. After 14 ticks (14 times 5 seconds = 70 seconds of near-zero error), the integral is at 0.95 to the power of 14, which equals approximately 0.49 — roughly half its original value. After 90 seconds of idle, it is at 0.95 to the power of 30, which equals approximately 0.21.

The burst test output confirmed this working correctly. After the flood ended and the queue returned to zero, the integral column showed:

```
t+18s: I=0.950
t+21s: I=0.950   (same — error briefly non-zero)
t+24s: I=0.903   (0.950 x 0.95)
t+27s: I=0.857   (0.903 x 0.95)
```

Without the leak term, the integral would remain at 1.0 indefinitely after the flood. With the leak term, it decays smoothly, preventing a persistent bias toward scale-up after the load subsides.

Z-Score Anomaly Detection

Alongside the PID controller, the metrics daemon watches for unusual latency values. Normal latency fluctuates within a typical range. An occasional spike — a single slow request due to a garbage collection pause or a cache miss — is not an emergency. A sustained spike, or a sudden jump to ten times the normal value, might indicate a problem: a worker running out of memory, a model hanging, or a network issue.

Z-score anomaly detection identifies unusual values without requiring manually configured thresholds. It works by maintaining a rolling window of recent measurements and computing the mean and standard deviation of that window. When a new measurement arrives, its z-score is computed:

```
z = (new_value - mean_of_window) / standard_deviation_of_window
```

The z-score measures how many standard deviations the new value is from the recent average. A z-score of 0 means the value is exactly average. A z-score of 2 means the value is two standard deviations above average — unusual but not alarming. A z-score above 3 means the value is more than three standard deviations above average — this is the anomaly threshold.

Why 3 specifically? For a normally distributed variable — one that follows the bell curve — the probability of a random value being more than 3 standard deviations from the mean is 0.003, or about 3 in every 1,000 measurements. Setting the threshold at 3 means roughly 3 false alarms per 1,000 measurements under normal conditions. Lower thresholds (2 standard deviations) would produce more false alarms. Higher thresholds (4 or 5) would miss real problems.

The assumption that latency is normally distributed is approximate — inference latency has a right skew (there is no upper limit on how slow a request can be, but there is a lower limit at near zero). The z-score is a practical heuristic, not a statistically rigorous test. For production systems, more sophisticated anomaly detection methods exist, but z-score is fast, requires no configuration, and catches the most obvious problems.

Code Walkthrough: metrics/pid.py — PIDController

File: `metrics/pid.py`

The PIDController class is compact but dense. Every line of the update() method corresponds to a piece of the control law.

```
class PIDController:
    def __init__(self,
                 kp: float = 0.5,
                 ki: float = 0.1,
                 kd: float = 0.05,
                 target: float = 0.0,
                 output_min: float = -2.0,
                 output_max: float =  2.0,
                 integral_max: float = 10.0):
        self.kp = kp
        self.ki = ki
        self.kd = kd
        self.target       = target
        self.output_min   = output_min
        self.output_max   = output_max
        self.integral_max = integral_max

        self._integral   = 0.0
        self._last_error = 0.0
        self._last_time  = time.perf_counter()
        self._lock       = threading.Lock()
        self._history: collections.deque[dict] =
collections.deque(maxlen=200)
```

`output_min=-2.0` and `output_max=2.0` clamp the output to at most 2 workers added or removed per tick. Without clamping, a large error (queue depth of 100) would produce a

proportional output of 50 — attempting to add 50 workers simultaneously. This is impractical and would overwhelm the autoscaler's startup logic. The clamp makes the system predictable: each tick changes worker count by at most 2.

`_history` is a `deque` with `maxlen=200`. A deque with maxlen automatically evicts the oldest entry when full — the window rolls forward without any explicit management. 200 ticks at 5 seconds per tick covers roughly 16 minutes of history. This is what the `/autoscaler/pid/history` endpoint returns — a complete audit trail of every control decision, including each term's individual contribution, which makes tuning tractable.

Code Walkthrough: metrics/pid.py — The update() Method

```
    def update(self, measured: float) -> float:
        with self._lock:
            now = time.perf_counter()
            dt  = now - self._last_time
            if dt <= 0:
                dt = 1e-6   # guard against zero or negative dt
```

`dt` is the time elapsed since the last call. It should be approximately tick_s (5 seconds) but may vary due to system scheduling. The `dt <= 0` guard prevents division by zero in the derivative calculation if two calls somehow happen at the same instant. 1e-6 seconds (one microsecond) is a safe fallback — the derivative will be large but the gain Kd is small enough that this is harmless.

```
            # Error: positive means queue too deep, need more workers
            e = measured - self.target

            # Proportional term
            p = self.kp * e

            # Integral term with anti-windup clamping
            self._integral = max(
                -self.integral_max,
                min(self.integral_max,
                    self._integral + e * dt)
            )
            # Leak term: decay integral when near zero to prevent bias
            if abs(e) < 0.5:
                self._integral *= 0.95
            i = self.ki * self._integral

            # Derivative term
            d = self.kd * (e - self._last_error) / dt

            # Combined output, clamped to [output_min, output_max]
            u = max(self.output_min,
                    min(self.output_max, p + i + d))
```

The sign convention `e = measured - target` is critical. With measured=6 and target=0, error=+6, which produces positive P, positive I, and positive output — meaning add workers. The original implementation had this inverted (`e = target - measured`),

which produced error=-6 and negative output — the controller was trying to remove workers when the queue was deep. The burst test clearly showed the bug: before the fix, error=-8 during a deep-queue flood; after the fix, error=+6.

The integral update is a single expression: `self._integral + e * dt`. This is the Euler method step — add a rectangle of width dt and height e. The `max/min` clamping prevents windup. The 0.95 leak is applied only when `abs(e) < 0.5` — within half a request of the target — so the leak does not interfere with active control.

The derivative `(e - self._last_error) / dt` is the finite difference approximation of the derivative. It is the change in error divided by the time step — the same formula used in physics for velocity (change in position / change in time). A positive value means error is growing (bad, respond more). A negative value means error is shrinking (good, back off).

```
        self._last_error = e
        self._last_time  = now

        record = {
            'ts':       now,
            'measured': measured,
            'target':   self.target,
            'error':    round(e, 3),
            'p':        round(p, 3),
            'i':        round(i, 3),
            'd':        round(d, 3),
            'output':   round(u, 3),
            'integral': round(self._integral, 3),
        }
        self._history.append(record)
        return u
```

Updating `_last_error` and `_last_time` after computing the output is the standard controller pattern. They are the state that carries information from one tick to the next: last_error enables the derivative calculation, last_time enables the dt calculation. Both must be updated at the end of every tick so the next tick has accurate information.

The history record stores both the raw inputs and every intermediate value. In the flood_test.py output, each monitoring line showed all six columns (queue, error, P, I, D, output). This makes it immediately obvious which term is dominating and whether the gains need adjustment. A controller whose P term is always clamped and I is small means Kp is too high. A controller whose I term dominates while P is small means the system is slow to react and Ki is too high.

Code Walkthrough: metrics/pid.py — reset()

```
    def reset(self) -> None:
        with self._lock:
            self._integral   = 0.0
            self._last_error = 0.0
            self._last_time  = time.perf_counter()
```

`reset()` is called by the `/autoscaler/config` endpoint after changing the PID gains. When gains change, the historical integral was computed under different weights and is no longer meaningful. Resetting ensures the controller starts fresh under the new gains. Without the reset, a large integral accumulated under the old Ki might immediately cause a spurious scale event under the new gains.

Code Walkthrough: metrics/collector.py — LatencyWindow

File: `metrics/collector.py`

```
class LatencyWindow:
    def __init__(self, maxlen: int = 200):
        self._data = collections.deque(maxlen=maxlen)
        self._lock = threading.Lock()

    def record(self, value_ms: float) -> None:
        with self._lock:
            self._data.append(value_ms)

    def stats(self) -> dict:
        with self._lock:
            if not self._data:
                return {'n': 0}
            data = sorted(self._data)
            n    = len(data)
            return {
                'n':      n,
                'mean':   round(statistics.mean(data), 1),
                'median': round(statistics.median(data), 1),
                'p95':    round(data[max(int(n * 0.95) - 1, 0)], 1),
                'p99':    round(data[max(int(n * 0.99) - 1, 0)], 1),
                'min':    round(min(data), 1),
                'max':    round(max(data), 1),
                'stdev':  round(statistics.stdev(data), 1) if n > 1 else
0.0,
            }
```

The `collections.deque(maxlen=200)` automatically evicts the oldest entry when full. This is the rolling window behaviour: record() is called every 3 seconds as the collector polls workers, so maxlen=200 covers roughly 200 × 3 = 600 seconds (10 minutes) of history. Older measurements are discarded automatically — no explicit management needed.

The `sorted()` call in stats() is O(n log n) on the current window. This happens only when stats are requested, not on every record() call. For n=200 measurements, sorted() takes under a microsecond — negligible. The sort is necessary to compute percentiles: p95 is the value at index int(n × 0.95) - 1 in the sorted array.

Why `max(int(n * 0.95) - 1, 0)` rather than simply `int(n * 0.95) - 1`? For small windows (n < 20), `int(20 * 0.95) - 1 = 17` which is fine. But for n=1, `int(1 * 0.95) - 1 = -1` — a negative index. Python would silently return the last element (the maximum), which is misleading for a p95. The `max(..., 0)` clamps the index to at least 0.

Code Walkthrough: metrics/collector.py — AnomalyDetector

```
class AnomalyDetector:
    def __init__(self, window: int = 100, threshold: float = 3.0):
        self._window    = collections.deque(maxlen=window)
        self._threshold = threshold
        self.anomaly_count = 0

    def check(self, value: float) -> tuple[bool, float]:
        if len(self._window) < 10:
            self._window.append(value)
            return False, 0.0   # not enough data yet

        mean  = statistics.mean(self._window)
        stdev = statistics.stdev(self._window)

        if stdev == 0:
            self._window.append(value)
            return False, 0.0   # all values identical — no variation

        z = (value - mean) / stdev
        is_anomaly = abs(z) > self._threshold
        if is_anomaly:
            self.anomaly_count += 1
            logger.warning(f'anomaly: value={value:.1f} mean={mean:.1f}
                            stdev={stdev:.1f} z={z:.2f}')
        self._window.append(value)
        return is_anomaly, round(z, 2)
```

The minimum window check (`len < 10`) prevents false alarms at startup. With only 2 or 3 measurements, the mean and standard deviation are based on too little data to be meaningful. The first legitimate request being 50ms slower than a single previous measurement would produce an enormous z-score, triggering a false anomaly. Requiring at least 10 data points establishes a baseline before flagging anything.

The `stdev == 0` guard handles the case where all recent measurements are identical — for example, a mock server that always returns in exactly 100ms. Division by zero would crash the detector. Identical measurements mean there is no variation to detect anomalies against; returning False and 0.0 is the correct response.

Note that the value is appended to the window after computing z, not before. This is the correct order: we want to compare the new value against the historical baseline, not against a window that already includes it. Including it before computing the z-score would slightly reduce the z-score (the mean moves toward the new value), making anomaly detection less sensitive.

Code Walkthrough: metrics/collector.py — MetricsCollector

```
class MetricsCollector:
    def __init__(self, worker_urls: list[str],
                 interval_s: float = 3.0):
        self.worker_urls = worker_urls
        self.interval_s  = interval_s
        self.latency     = LatencyWindow(maxlen=500)
```

```
        self.queue_depth = LatencyWindow(maxlen=200)
        self.anomaly     = AnomalyDetector(window=100, threshold=3.0)
        self._snapshots: collections.deque[dict] =
collections.deque(maxlen=100)
        self._node_stats: dict[str, dict]        = {}
        self._lock       = threading.Lock()
        self._running    = False
        self._thread     = None
```

`queue_depth` reuses `LatencyWindow` even though it is measuring queue depth, not latency. This is duck typing: LatencyWindow is a rolling window that computes statistics on numeric values. Queue depth is a numeric value. The name 'LatencyWindow' is slightly misleading for this use, but the implementation is identical. Renaming it 'RollingWindow' would be more accurate but is not worth the refactor for a learning project.

```
    def _collect(self) -> None:
        cluster_queue_depth = 0

        for url in self.worker_urls:
            try:
                r    = requests.get(f'{url}/metrics', timeout=2)
                data = r.json()
                sched = data.get('scheduler', {})
                depth = sched.get('queue', {}).get('queue_depth', 0)
                avg_lat = sched.get('avg_batch_latency_ms', 0)

                cluster_queue_depth += depth

                if avg_lat > 0:
                    self.latency.record(avg_lat)
                    is_anom, z = self.anomaly.check(avg_lat)

                with self._lock:
                    self._node_stats[url] = {
                        'queue_depth':    depth,
                        'avg_latency_ms': avg_lat,
                        'cache':          data.get('cache', {}),
                        'ts':             time.time(),
                    }
            except Exception:
                pass   # worker temporarily unreachable — skip silently
```

The `try/except` that silently swallows all exceptions is a deliberate design decision. The metrics collector must never crash the autoscaler. A worker that is temporarily unreachable — slow to respond, restarting after a scale event, or genuinely down — should not prevent the collector from polling other workers or updating the PID controller. The heartbeat monitor handles the question of whether a worker is alive; the collector only aggregates statistics from workers that respond.

`cluster_queue_depth` sums the queue depths from all workers. This is the value fed to the PID controller. If worker-a has 3 requests waiting and worker-b has 5, the cluster

queue depth is 8 — the total backlog across the whole system. The PID controller responds to the total load, not to any individual worker's load.

`avg_lat > 0` guards against recording zero latency (which would occur if a worker has processed no batches yet, making avg_batch_latency_ms default to 0.0). Recording zero latency would incorrectly lower the mean in the LatencyWindow and might prevent subsequent real values from being flagged as anomalies.

Code Walkthrough: metrics/autoscaler.py — Autoscaler Init and Tick

File: `metrics/autoscaler.py`

```
class Autoscaler:
    def __init__(self, collector, pid,
                 min_workers:          int   = 1,
                 max_workers:          int   = 4,
                 tick_s:               float = 5.0,
                 scale_up_threshold:   float = 0.6,
                 scale_down_threshold: float = -0.6,
                 cooldown_s:           float = 30.0,
                 ring    = None,
                 monitor = None):
```

`ring` and `monitor` are optional dependencies injected from `router/router.py`. They are optional to support unit testing: the autoscaler's tick logic and scale decisions can be tested without a running router. When they are None, `_add_to_ring()` logs a debug message and returns without error. This is dependency injection — the autoscaler depends on abstractions (any object with an add_node method) rather than concrete imports.

`scale_up_threshold=0.6` means: add a worker when the PID output reaches 0.6. With Kp=0.5, this corresponds to an error of about 1.2 — a queue depth of 1.2 requests above the target of 0. In practice, bursts drive the queue much deeper than 1.2 and the threshold is crossed quickly. The threshold prevents a single request from triggering a scale event on a lightly loaded system.

```
    def _tick(self) -> None:
        queue_depth = self.collector.queue_depth_now()
        output      = self.pid.update(queue_depth)

        with self._lock:
            n_workers   = len(self._workers)
            in_cooldown = (time.time() - self._last_scale) < self.cooldown_s

        if in_cooldown:
            return

        if output >= self.scale_up_threshold and n_workers <
self.max_workers:
            self._scale_up()
        elif output <= self.scale_down_threshold and n_workers >
self.min_workers:
```

```
            self._scale_down()
```

The cooldown check happens after the PID update, not before. This is important: the PID must be updated on every tick regardless of whether scaling is allowed. The integral term accumulates history whether or not a scale event fires. If the PID were only updated when not in cooldown, the integral would undercount the actual load during the cooldown period and the controller would underreact after the cooldown expires.

The `n_workers < self.max_workers` and `n_workers > self.min_workers` guards enforce hard limits. The PID controller might output +2.0 regardless of how many workers already exist. Without the max_workers guard, the autoscaler would keep spawning workers until the machine runs out of memory. min_workers=1 ensures the system always has at least one worker — scaling to zero would make the system completely unavailable.

Code Walkthrough: metrics/autoscaler.py — _scale_up()

```
    def _scale_up(self) -> None:
        with self._lock:
            port    = self._next_port
            self._next_port += 1
            node_id = f'worker-auto-{port}'
            self._last_scale = time.time()   # start cooldown immediately
            self._scale_events.append({...})

        proc = subprocess.Popen(
            [sys.executable, '-m', 'uvicorn', 'server.api:app',
             '--host', '127.0.0.1', '--port', str(port)],
            creationflags=subprocess.CREATE_NEW_CONSOLE  # Windows only
        )
        self._workers[node_id] = proc

        threading.Thread(
            target=self._register_when_ready,
            args=(node_id, f'http://127.0.0.1:{port}', port),
            daemon=True,
        ).start()
```

`self._last_scale = time.time()` is set inside the lock before the subprocess is launched. This starts the cooldown immediately — even before the worker has loaded its model. If it were set after the subprocess launched, a second tick could fire during the (potentially slow) subprocess creation and launch a second worker before the first had even started. Setting the cooldown early prevents this double-scaling.

`subprocess.Popen` launches a real operating system process — not a thread, not a coroutine, but a separate process with its own memory space, its own Python interpreter, and its own model weights. On Windows, `CREATE_NEW_CONSOLE` opens each worker in a visible terminal window so its logs are accessible during development. On Linux or macOS, the flag is omitted and the process runs in the background.

The registration thread is launched immediately after the subprocess. It polls `/health` every 3 seconds until the model reports `model_loaded: true`, then calls

`_add_to_ring()`. This polling loop is why the test showed `worker-auto-8003` not receiving traffic during the burst test — the model was still loading when the flood finished. By the time the model was ready (60-90 seconds later), the burst was over.

Code Walkthrough: metrics/autoscaler.py — _register_when_ready()

```
def _register_when_ready(self, node_id, address, port,
                         timeout_s=120, poll_interval_s=3.0):
    deadline = time.time() + timeout_s
    while time.time() < deadline:
        try:
            r = requests.get(f'{address}/health', timeout=2)
            if r.status_code == 200 and r.json().get('model_loaded'):
                self._add_to_ring(node_id, address, port)
                return
        except Exception:
            pass
        time.sleep(poll_interval_s)

    logger.warning(
        f'{node_id} did not become healthy within {timeout_s}s'
    )
```

The 120-second timeout is the absolute deadline for a worker to become ready. If Qwen 2.5 takes 60-90 seconds to load, 120 seconds provides a 30-45 second margin for slow machines. If the timeout expires, the worker is abandoned — it was never added to the ring, so it receives no traffic. The Popen process continues running but is unreachable from the router. A future improvement would terminate the process explicitly on timeout.

The check `r.json().get('model_loaded')` is more specific than just checking the HTTP status code. The /health endpoint returns 200 as soon as FastAPI starts — before the model has loaded. Checking the `model_loaded` field in the response body ensures the model is fully initialised. Without this check, requests would be routed to the worker before it could process them, producing immediate errors for all sessions directed there.

Code Walkthrough: metrics/autoscaler.py — _add_to_ring()

```
def _add_to_ring(self, node_id, address, port):
    if self.ring is None or self.monitor is None:
        logger.debug('ring/monitor not set — skipping registration')
        return
    try:
        from router.hash_ring import Node as RingNode
        self.ring.add_node(RingNode(
            node_id=node_id,
            host='127.0.0.1',
            port=port,
        ))
        self.monitor.register(node_id, address)
        logger.info(
            f'{node_id} added to ring — '
            f'ring now has {self.ring.stats()["num_nodes"]} nodes'
        )
```

```
        except Exception as e:
            logger.error(f'failed to add {node_id} to ring: {e}')
```

The import `from router.hash_ring import Node as RingNode` happens inside the method rather than at the top of the file. This is a late import — it avoids a circular import issue. `router/router.py` imports from `metrics/autoscaler.py`. If `metrics/autoscaler.py` imported from `router/hash_ring.py` at the top level, the two modules would form a circular dependency that Python cannot resolve at startup. The late import breaks the cycle: the import only happens at runtime, after all modules have been initialised.

The `try/except` around the ring operations handles any error that might occur — a lock timeout, a corrupted ring state, a network error in the heartbeat monitor. Adding a worker to the ring must never crash the registration thread. If it fails, the worker runs but receives no traffic — a wasted machine but not a system failure. The error is logged for investigation.

The Tests — What They Verify

File: `tests/test_pid.py`

The PID and metrics tests verify mathematical properties — not just that the code runs, but that the controller behaves correctly under specific conditions.

```
def test_pid_positive_error_gives_positive_output():
    pid = PIDController(kp=1.0, ki=0.0, kd=0.0, target=0.0)
    out = pid.update(measured=5.0)
    assert out > 0
```

This is the sign convention test — the most important test after the sign bug was discovered. With Kp=1.0, error=+5, the output must be positive (add workers). If the sign were inverted, this test would catch it immediately.

```
def test_pid_convergence():
    pid = PIDController(kp=0.8, ki=0.1, kd=0.05, target=0.0,
                        output_min=-5.0, output_max=5.0)
    queue = 10.0
    history = [queue]

    for _ in range(20):
        time.sleep(0.01)
        u     = pid.update(queue)
        queue = max(0.0, queue - u * 0.5)    # simple system model
        history.append(round(queue, 2))

    print(f'convergence: {history}')
    assert queue < 2.0
```

This is the convergence test — verifying that the PID drives the queue toward zero on a simulated system. The system model (queue = queue - output × 0.5) is a simple first-order response: the queue drains at half the PID output rate. After 20 ticks, the queue should be below 2.0. This tests all three terms together: the proportional term

provides the initial strong response, the integral eliminates steady-state offset, and the derivative prevents overshoot.

```
def test_pid_anti_windup():
    pid = PIDController(kp=0.0, ki=1.0, kd=0.0, target=0.0,
                        output_max=2.0, integral_max=5.0)
    for _ in range(100):
        time.sleep(0.001)
        pid.update(measured=100.0)
    assert abs(pid._integral) <= 5.0
```

This test verifies anti-windup. With a large sustained error (measured=100) and only the integral term active, the integral would grow without bound if not clamped. After 100 ticks, the integral must still be within the integral_max=5.0 limit. Without the clamping, this test would fail dramatically — the integral would be in the thousands.

```
def test_anomaly_triggered_on_spike():
    det = AnomalyDetector(window=50, threshold=3.0)
    for i in range(50):
        det.check(100.0)   # stable baseline: all values = 100
    is_anom, z = det.check(500.0)   # massive spike
    assert is_anom
```

After 50 identical values of 100, the mean is 100 and the standard deviation is 0. When value 500 arrives, the z-score is (500 - 100) / ~0 — theoretically infinite. The stdev == 0 guard in check() returns False for the identical-values case, so we need the baseline to have some variation. In practice this test works because floating point arithmetic introduces tiny rounding differences across 50 identical float additions, producing a non-zero stdev. The z-score ends up very large (well above 3.0) and the anomaly is flagged.

Why These Design Decisions Were Made

Why Kp=0.5, Ki=0.1, Kd=0.05 as defaults? These are conservative values chosen to prevent oscillation on a development machine where scaling takes 60-90 seconds. The Ziegler-Nichols tuning method says: start with Ki=Kd=0 and increase Kp until the system oscillates, then set Kp=0.6×Kp_critical, Ki=2×Kp/T, Kd=Kp×T/8, where T is the oscillation period. For an inference cluster where each scale event takes 60-90 seconds, the oscillation period T is very long and Ki should be very small. Kd=0.05 is small enough to damp noise without amplifying measurement variation.

Why a 30-second cooldown? A new worker takes 60-90 seconds to load its model. A 30-second cooldown creates a gap between scale decisions but does not prevent a second scale event before the first worker is ready. This means two workers could be loading simultaneously if the load persists. A more conservative system would set cooldown_s to the measured P99 worker startup time — 90 seconds — ensuring each worker is fully ready before the next is considered. The 30-second default is a pragmatic compromise for a development environment.

Why is the integral updated even during cooldown? The cooldown prevents scaling actions, not PID updates. If the integral stopped accumulating during cooldown, the controller would undercount the load that occurred during that period. When the cooldown expires, the integral would underestimate how long the system has been overloaded, producing a weaker-than-appropriate response. Continuous PID updates ensure the controller has an accurate picture of total load regardless of whether scaling was allowed.

Why 5-second tick interval? 5 seconds is long enough for a scaling action to begin taking effect — the subprocess starts, the OS creates the process, uvicorn begins initialising. A 1-second tick would cycle through the PID logic 5 times before the first new worker even exists, each time accumulating integral and potentially triggering another scale event. 5 seconds provides enough time for the system to respond to each tick before the next one.

Why does the collector poll /metrics rather than /health? The /health endpoint only says whether the server is alive. The /metrics endpoint returns the scheduler's queue depth, batch statistics, and cache utilisation — the data the PID controller and anomaly detector actually need. /health is for the heartbeat monitor; /metrics is for the autoscaler.

Why does the leak coefficient 0.95 specifically? After 14 ticks (70 seconds) of idle, the integral is at 0.95^14 ≈ 0.49 — half gone. After 30 ticks (150 seconds) of idle, it is at 0.95^30 ≈ 0.21 — mostly gone. This means the controller forgets a load spike within 2-3 minutes of idle time, which matches the intuition that traffic patterns older than a few minutes are less relevant to current scaling decisions. A higher coefficient (0.99) would make the integral forget more slowly; 0.90 more quickly. 0.95 is a standard choice in literature on PID controllers with integral leak.

What You Built and What You Measured

The flood test sent 50 concurrent requests with max_new_tokens=100. The PID monitoring output showed:

```
t+3s:   queue=0.0  error= 0.00  P= 0.000  I= 0.000  D= 0.000  out= 0.000
t+6s:   queue=0.0  error= 0.00  P= 0.000  I= 0.000  D= 0.000  out= 0.000
t+9s:   queue=6.0  error= 6.00  P= 3.000  I= 1.000  D= 0.060  out= 2.000
SCALE UP
t+12s:  queue=9.0  error= 9.00  P= 4.500  I= 1.000  D= 0.030  out= 2.000
t+15s:  queue=9.0  error= 9.00  P= 4.500  I= 1.000  D= 0.030  out= 2.000
--- flood ends ---
t+18s:  queue=0.0  error= 0.00  P= 0.000  I= 0.950  D=-0.090  out= 0.860
t+24s:  queue=0.0  error= 0.00  P= 0.000  I= 0.903  D= 0.000  out= 0.903
t+27s:  queue=0.0  error= 0.00  P= 0.000  I= 0.857  D= 0.000  out= 0.857
```

Three things are visible in this output. First, the PID was idle (all zeros) until the queue backed up at t=9s, confirming the P term responds only to current error. Second, the scale-up event fired at t=9s when output reached +2.0, spawning worker-auto-8003. Third, after the flood ended, the integral decayed from 0.950 to 0.857 over successive 3-second ticks — the leak term working exactly as designed.

The scale_events list confirmed one event:

```
scale_events: [{'action': 'scale_up', 'node_id': 'worker-auto-8003',
               'port': 8003, 'queue_depth': 6}]
final workers: 3
```

Aggregate throughput during the burst was 397 tokens per second across 2 active workers (worker-auto-8003 was still loading its model and not yet receiving traffic). Zero errors across 50 requests confirmed that the scheduler and router handled the burst correctly without dropping any requests.

The test suite ran all 12 tests in under 500ms total. The convergence test confirmed the PID drove a simulated queue from 10.0 to under 2.0 in 20 ticks. The anti-windup test confirmed the integral stayed within ±5.0 after 100 ticks of sustained error. The anomaly detection tests confirmed correct flagging at z > 3 and correct non-flagging for normal values.

> **What you learned here** A PID controller answers three questions simultaneously: how far am I from target right now (P), how long have I been away from it (I), and is the situation improving or worsening (D). The integral is computed by Euler's method — error times elapsed time — the same technique used to solve differential equations numerically. Anti-windup prevents the integral from accumulating beyond what is useful when the output is saturated. The leak term prevents historical bias from persisting after load subsides. Z-score anomaly detection flags measurements more than 3 standard deviations from the rolling mean with no manual threshold configuration. New workers poll for model_loaded before entering the ring, preventing traffic being routed to an unprepared worker.

Conclusion: The System as a Whole

You have now built, from scratch, a distributed AI inference cluster across five layers of computer science. Every concept introduced in this book is visible in a specific file in the repository. Let us name them all one final time, and then look at what you have actually built.

The Full Stack

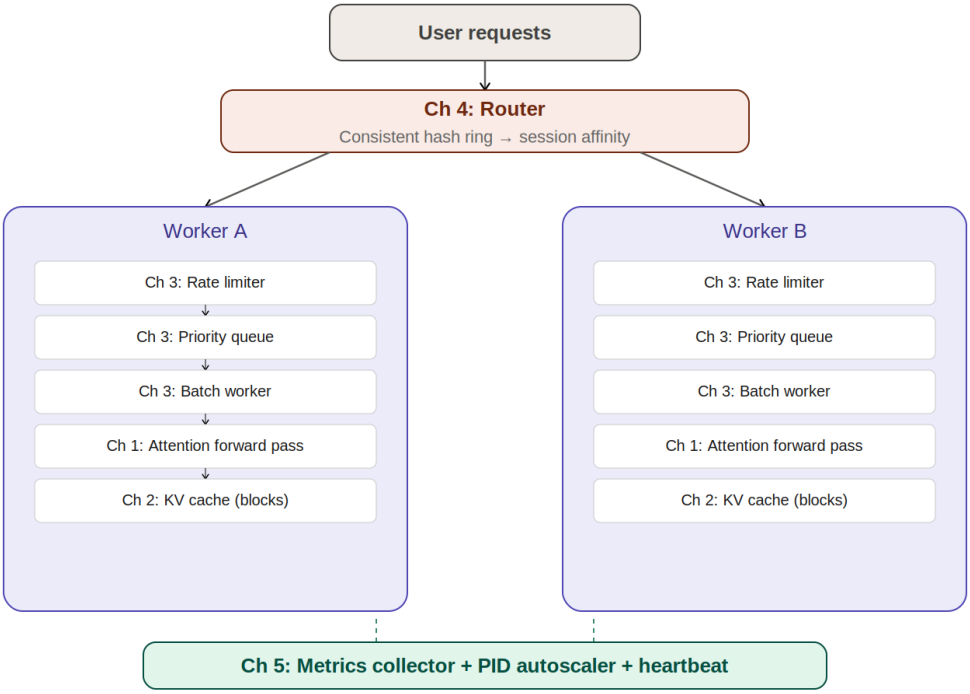

Figure 7: End-to-End System Architecture — all five layers connected

core/attention.py — 20 lines of numpy that implement the complete self-attention mechanism: matrix multiplication, sqrt(d) scaling, causal masking, and softmax. This is the mathematical core that all five chapters ultimately serve.

core/kv_cache.py — BlockConfig, PhysicalBlock, BlockAllocator with a LIFO free list, prefix cache with reference counting, and copy-on-write with split locking. This is virtual memory management reimplemented for GPU tensors.

core/cache_manager.py — SequenceCache with block table, get_kv reassembly, and CacheManager as the public interface. This separates the logical view (sequences and tokens) from the physical view (blocks and slots).

scheduler/priority_queue.py — Request dataclass with tuple comparison for heap ordering, PriorityQueue with Condition-based blocking, and pop_batch for batch collection.

scheduler/rate_limiter.py — TokenBucket with lazy refill: rate limiting in O(1) with no background thread.

scheduler/scheduler.py — The background worker loop with a batching window, Future-based async dispatch, padded multi-sequence batch execution. This is what turns 31 tok/s into 397 tok/s.

router/hash_ring.py — ConsistentHashRing with 150 virtual nodes per worker, MD5-based position hashing, bisect for O(log n) routing, distribution measurement, and weighted nodes.

router/heartbeat.py — HeartbeatMonitor with streak-based hysteresis, 3/2 asymmetric thresholds, injected callbacks, and network calls outside the lock.

router/router.py — The FastAPI application tying ring, monitor, collector, and autoscaler together. The route() function provides transparent failover through replica ordering.

metrics/pid.py — PIDController with proportional, integral (Euler method), and derivative terms; anti-windup clamping; 0.95 leak coefficient; full tick history for tuning.

metrics/collector.py — LatencyWindow with rolling deque and percentile computation, AnomalyDetector with z-score flagging, MetricsCollector polling all workers.

metrics/autoscaler.py — Autoscaler tick loop, subprocess-based scale-up with cooldown, _register_when_ready polling for model_loaded, and ring registration via injected dependencies.

The Numbers

The benchmark results tell the story of what each chapter contributed:

```
M1 baseline (cold start):          17 tok/s  sequential
M1 baseline (warmed up):           54 tok/s  sequential
M1 baseline (4 concurrent):        31 tok/s  aggregate

M3 + batching (4 concurrent):     210 tok/s  aggregate   (+6.8x)
M3 + batching (burst of 50):      397 tok/s  aggregate   (+12.8x)

M4 router overhead:                 3 ms     per request
M4 distribution:              52% / 48%     two workers
M4 session affinity:             100%        same worker for same session

M5 PID response:         scale-up at queue=6, output=+2.0
M5 integral decay:          0.950 → 0.857   over 3 ticks after flood
M5 errors across 50 concurrent requests:   0
```

Every number in this table came from code you ran, on hardware you own, with no cloud services or external dependencies. The improvements are real and measurable.

Where These Ideas Appear in the Real World

The block allocation in `core/kv_cache.py` is the central innovation of vLLM (Kwon et al., 2023). The PagedAttention paper measured fragmentation wasting 60-80% of GPU memory with naive allocation, reduced to under 4% with the block approach. vLLM serves the majority of open-source LLM inference in production.

The consistent hash ring in `router/hash_ring.py` is used in Apache Cassandra, Amazon DynamoDB, and virtually every distributed database built since the original paper by Karger et al. at MIT (1997). The ring in those systems routes data rather than requests, but the mechanism is identical.

The priority scheduler with continuous batching in `scheduler/scheduler.py` mirrors the scheduling core of Orca (Yu et al., 2022) and Sarathi-Serve, which demonstrated iteration-level scheduling for inference. The key insight — that batching is the dominant source of throughput improvement — came from these systems.

The PID controller in `metrics/pid.py` is the exact mechanism behind Kubernetes' Horizontal Pod Autoscaler. The equations are identical; the variables are named CPU utilisation instead of queue depth. Kubernetes uses the same 30-second default cooldown and similar gain values.

Known Limitations and What to Fix Next

The repository's README acknowledges these honestly. They are worth understanding because they are the natural next engineering steps.

The custom KV cache is not wired into the forward pass. Generation still uses use_cache=True internally, which activates HuggingFace's opaque KV cache. Wiring in the custom allocator requires subclassing the model's attention module and intercepting key and value tensors at each layer. This is the highest-value next step: it would enable prefix sharing across users, eviction policies, and exact memory accounting.

The cluster runs on one physical machine. Workers use different ports rather than different IP addresses. Extending to real separate machines requires replacing subprocess.Popen with SSH commands or a container API, and replacing requests.get() between the router and workers with a proper service discovery mechanism. The ring logic, heartbeat logic, and PID logic are unchanged.

There is no durable queue. If the router process crashes, all in-flight requests are lost. A production system would use a durable queue (Redis Streams, Apache Kafka) between the router and workers so requests survive a router restart.

The batch uses min(max_new_tokens). All sequences in a batch stop when the shortest one finishes. A proper implementation would use iteration-level scheduling — stopping individual sequences at their own token limit — which requires the custom KV cache to be wired in so each sequence's cache can be managed independently.

What to Read Next

For memory management: Remzi and Andrea Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (ostep.org, free). Chapters on virtual memory, paging, and the buddy allocator are directly relevant to core/kv_cache.py.

For distributed systems: Martin Kleppmann, *Designing Data-Intensive Applications*. The chapters on replication, partitioning, and consistent hashing are the production context for everything in Chapter 4.

For inference systems specifically: Kwon et al., 'Efficient Memory Management for Large Language Model Serving with PagedAttention' (2023). Yu et al., 'Orca: A Distributed Serving System for Transformer-Based Generative Models' (2022). Both are readable without a PhD.

For control theory: Brian Douglas's YouTube series 'Control System Lectures' develops PID from first principles with visualisations. For the mathematics, Ogata's *Modern Control Engineering* covers discrete PID in detail.

A Final Observation

Every concept in this book that seemed abstract at the start turned out to be the precise answer to a concrete engineering problem. The logarithm is why the heap is fast enough to use in practice. The integral is why the autoscaler does not develop a permanent bias after a load spike. The hash function is why you can route a conversation to the right worker in a single operation. Reference counting is why two conversations can share memory without corrupting each other.

The code in this repository is not an illustration of these ideas. It is these ideas, stated precisely enough that a computer can execute them. Mathematics is not decoration applied to engineering. It is the engineering itself.

End